

Automated Sandbox-Based System and Method for Detecting Malicious Behaviour in Executable Files

A PROJECT REPORT

Major Project – II (01CE0807)

Submitted by

ADITYA
92200103007

TETAR MILAN BHARATBHAI
92200103022

AADITYASINH JADEJA
92200103105

BACHELOR OF TECHNOLOGY

in

Computer Engineering



Faculty of Engineering & Technology

Marwadi University, Rajkot

January, 2026



Major Project-II (01CE0807)

Department of Computer Engineering

Faculty of Engineering & Technology

Marwadi University

A.Y. 2025-26

CERTIFICATE

This is to certify that the project report submitted along with the project entitled **Automated Sandbox-Based System and Method for Detecting Malicious Behaviour in Executable Files** has been carried out by **ADITYA** (92200103007), **Tetar Milan Bharatbhai** (9200103022), **Aadityasinh Jadeja** (92200103105) under my guidance in partial fulfilment for the degree of Bachelor of Technology in Computer Engineering, 8th Semester of Marwadi University, Rajkot during the academic year 2025-26.

Prof. Yogeshwar Prajapati

Assistant Professor

Department of Computer Engineering

Prof. (Dr.) Krunal Vaghela

Associate Dean & Head

Department of Computer Engineering



Marwadi
University
Marwadi Chandarana Group



Major Project-II (01CE0807)

Department of Computer Engineering

Faculty of Engineering & Technology

Marwadi University

A.Y. 2025-26

CERTIFICATE

This is to certify that the project report submitted along with the project entitled **Automated Sandbox-Based System and Method for Detecting Malicious Behaviour in Executable Files** has been carried out by **ADITYA** (92200103007) under my guidance in partial fulfilment for the degree of Bachelor of Technology in Computer Engineering, 8th Semester of Marwadi University, Rajkot during the academic year 2025-26.

Prof. Yogeshwar Prajapati

Assistant Professor

Department of Computer Engineering

Prof. (Dr.) Krunal Vaghela

Associate Dean & Head

Department of Computer Engineering



Major Project-II (01CE0807)

Department of Computer Engineering

Faculty of Engineering & Technology

Marwadi University

A.Y. 2025-26

CERTIFICATE

This is to certify that the project report submitted along with the project entitled **Automated Sandbox-Based System and Method for Detecting Malicious Behaviour in Executable Files** has been carried out by **Tetar Milan Bharatbhai** (92200103022) under my guidance in partial fulfilment for the degree of Bachelor of Technology in Computer Engineering, 8th Semester of Marwadi University, Rajkot during the academic year 2025-26.

Prof. Yogeshwar Prajapati

Assistant Professor

Department of Computer Engineering

Prof. (Dr.) Krunal Vaghela

Associate Dean & Head

Department of Computer Engineering



Major Project-II (01CE0807)

Department of Computer Engineering

Faculty of Engineering & Technology

Marwadi University

A.Y. 2025-26

CERTIFICATE

This is to certify that the project report submitted along with the project entitled **Automated Sandbox-Based System and Method for Detecting Malicious Behaviour in Executable Files** has been carried out by **Aadityasinh Jadeja** (92200103105) under my guidance in partial fulfilment for the degree of Bachelor of Technology in Computer Engineering, 8th Semester of Marwadi University, Rajkot during the academic year 2025-26.

Prof. Yogeshwar Prajapati

Assistant Professor

Department of Computer Engineering

Prof. (Dr.) Krunal Vaghela

Associate Dean & Head

Department of Computer Engineering



Major Project-II (01CE0807)

Department of Computer Engineering

Faculty of Engineering & Technology

Marwadi University

A.Y. 2025-26

DECLARATION

We hereby declare that the **Major Project-II (01CE0807)** report submitted along with the Project entitled **Automated Sandbox-Based System and Method for Detecting Malicious Behaviour in Executable Files** submitted in partial fulfilment for the degree of Bachelor of Technology in Computer Engineering to Marwadi University, Rajkot, is a Bonafide record of original project work carried out by us at Marwadi University under the supervision of **Prof. Yogeshwar Prajapati** and that no part of this report has been directly copied from any students' reports or taken from any other source, without providing due reference.

	Name of the Student	Sign of Student
1	_____	_____
2	_____	_____
3	_____	_____

Acknowledgement

We would like to express our sincere gratitude to our internal guide, **Prof. Yogeshwar Prajapati**, for his invaluable guidance, continuous support, and insightful suggestions throughout the duration of this project. His expertise, encouragement, and constructive feedback were instrumental in successfully completing this work and overcoming various technical and conceptual challenges. We are also thankful to the faculty members of our department for providing the necessary resources and a conducive academic environment that facilitated the smooth progress of this project.

We extend our sincere appreciation to the **Head of the Department** for their valuable support, encouragement, and for providing an excellent academic platform that enabled us to carry out this project successfully.

Abstract

*The increasing sophistication and volume of malware pose a significant threat to modern computing systems, rendering traditional signature-based detection techniques ineffective against new and obfuscated attacks. To address this challenge, this project presents an **Automated Sandbox-Based System and Method for Detecting Malicious Behaviour in Executable Files**, focusing on dynamic analysis to identify previously unknown and zero-day threats. The proposed system employs an isolated sandbox environment to safely execute suspicious executable files and monitor their runtime behavior. By analyzing system-level activities such as file system modifications, registry changes, process creation, network communications, and API call patterns, the system generates detailed behavioral profiles of executables. These behavioral indicators are then evaluated using automated analysis rules and classification techniques to distinguish malicious behavior from benign execution. The system is designed with a modular architecture that enables full automation of sample submission, execution, monitoring, report generation, and result storage. This approach minimizes human intervention, improves analysis consistency, and enhances scalability. Emphasis is placed on security, isolation, and reproducibility to ensure that malware samples do not escape the controlled environment. Experimental evaluation demonstrates that the proposed sandbox-based approach is effective in detecting malicious behavior that static analysis methods often fail to identify. The system provides comprehensive analysis reports that aid security analysts in understanding malware intent and behavior. This project highlights the practical application of automated dynamic analysis in malware detection and contributes toward strengthening defensive cybersecurity mechanisms against evolving threats.*

List of Figures

Figure 2.2.1 Classification of Common Types of Malwares	7
Figure 2.2.2 Botnet Command-and-Control Architecture	8
Figure 2.2.3 Workflow of a Phishing Attack	8
Figure 2.3.1 Signature-Based Malware Detection Process	10
Figure 3.2.1 Distributed Cuckoo Sandbox Deployment Architecture.....	25
Figure 3.3.1 Net. Conf. of Host and Guest VM in Cuckoo Sandbox Environment.....	27
Figure 3.4.1 Ubuntu Sandbox Automation Workflow for Malware Analysis	31
Figure 3.6 Workflow of the Proposed Automated SB Mal Analysis Sys.....	34
Figure 3.8 Arch of the Proposed Automated Sandbox-Based Malware Analysis Sys.....	37
Figure 4.2 Cuckoo Sandbox Core Startup and Module Initialization	40
Figure 4.5.1 Cuckoo Sandbox Startup and Configuration Initialization	43
Figure 4.5.2 Malware Sample Submission to Cuckoo Sandbox	44
Figure 4.5.3 Malware Sample Execution and Report Generation Log	44
Figure 4.5.4 Automated Behaviour Report Extraction Script Execution	45
Figure 4.5.5 Generated Behavioural Verdict File After Rule-Based Detection	46
Figure 4.6 Cuckoo Sandbox Generated Analysis Artifacts Directory	48

List of Tables

Table 2.1 Traditional malware detection techniques.....	9
Table 2.7 Comparative Analysis of Existing Malware Analysis and Detection Techniques...	19
Table 3.2 Observation of Major System Components.....	26
Table 4.1 Tools and Technologies Used in Implementation.....	39

Abbreviations

API	Application Programming Interface
CLI	Command Line Interface
GUI	Graphical User Interface
JSON	JavaScript Object Notation
PE	Portable Executable
VM	Virtual Machine

Table of Contents

Acknowledgement.....	i
Abstract.....	ii
List of Figures.....	iii
List of Tables.....	iv
List of Abbreviations.....	v
Table of Contents.....	vi
CHAPTER 1 Introduction.....	1
1.1 Background and Motivation.....	1
1.2 Problem Statement.....	2
1.3 Objectives of the Project	3
1.4 Scope of the Project	3
1.5 Limitations	4
1.6 Report Organization	5
 CHAPTER 2 Literature Review	 6
2.1 Overview of Malware and Cyber Threat Landscape	6
2.2 Types of Malware and Attack Vectors	7
2.3 Traditional Malware Detection Techniques	9
2.3.1 Signature-Based Detection.....	10
2.3.2 Heuristic-Based Detection.....	11
2.3.3 Static Analysis Methods.....	12
2.4 Dynamic Malware Analysis	13
2.5 Sandbox-Based Analysis Techniques	13
2.6 Existing Sandbox Frameworks	14
2.6.1 Cuckoo Sandbox.....	15
2.6.2 Any.Run Sandbox.....	16
2.6.3 Hybrid Analysis.....	17
2.7 Research Gaps and Need for Automation	18

CHAPTER 3 System Design and Methodology	23
3.1 Proposed System Overview	23
3.2 System Architecture	24
3.3 Malware Analysis Lab Environment Design	26
3.3.1 Host System Configuration.....	27
3.3.2 Virtual Machine Setup.....	28
3.3.3 Network Configuration.....	29
3.4 Sandbox Automation Workflow	30
3.5 Sample Submission and Execution Process	31
3.6 Behavioral Data Collection Mechanism	32
3.7 Log Processing and Feature Extraction	35
3.8 Malicious Behaviour Detection Logic	35
 CHAPTER 4 Implementation and Experimental Results	 38
4.1 Tools and Technologies Used	38
4.2 Implementation of Cuckoo Sandbox Environment	39
4.3 Automation Script Development	40
4.4 Dataset and Sample Collection	41
4.5 Experiment Execution Procedure	42
4.6 Observed Behavioral Patterns	37
4.7 Result Analysis and Detection Accuracy	48
4.8 Performance Evaluation	49
 CHAPTER 5 Conclusion and Future Enhancements	 50
5.1 Conclusion	50
5.2 Proposed Future Enhancements	51
References.....	52

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

The rapid growth of digital technologies and widespread internet connectivity have significantly increased the dependency of individuals and organizations on computer systems. Along with these advancements, cyber threats have evolved in complexity, scale, and sophistication. Malicious software, commonly referred to as malware, has become one of the most critical security challenges, capable of causing data breaches, financial losses, system disruptions, and unauthorized access to sensitive information. Traditional security mechanisms such as firewalls and signature-based antivirus solutions are often insufficient to detect newly emerging or obfuscated malware, leading to an urgent need for more advanced detection techniques.

Malware authors continuously develop new methods to evade detection, including encryption, polymorphism, and code obfuscation. As a result, static analysis-based detection approaches, which rely on known signatures or predefined patterns, struggle to identify previously unseen malicious executables [14]. This limitation has led to the adoption of dynamic analysis techniques, where suspicious files are executed in controlled environments to observe their behavior. Sandbox-based analysis has proven to be an effective approach in this domain, as it allows real-time monitoring of file activities such as process creation, file system modifications, registry changes, and network communications without risking the host system. However, manual sandbox analysis is time-consuming and requires skilled security analysts to interpret behavioral reports. With the increasing volume of malware samples generated daily, there is a strong demand for automated systems that can analyze executable files efficiently and accurately. Automation in sandbox environments not only reduces human effort but also enhances detection speed and consistency. Motivated by these challenges, this project focuses on designing and implementing an automated sandbox-based system that executes suspicious executable files in an isolated environment, captures their behavioral characteristics, and assists in identifying malicious activities systematically.

The motivation behind this work is to contribute toward strengthening cybersecurity defenses by providing a scalable, automated, and reliable malware behavior detection framework. By

integrating sandbox automation and behavioral monitoring, the proposed system aims to support security researchers and organizations in detecting and analyzing modern malware threats effectively.

1.2 Problem Statement

The increasing sophistication and volume of malicious software pose a significant threat to modern computer systems and network infrastructures. Conventional malware detection techniques, primarily based on signature matching and static code analysis, are no longer sufficient to identify newly developed, polymorphic, or obfuscated malware. These advanced threats can easily bypass traditional antivirus systems, resulting in undetected infections and potential security breaches.

Dynamic analysis using sandbox environments has emerged as a reliable technique for understanding malware behavior by executing suspicious files in isolated virtual machines and monitoring their activities. However, existing sandbox-based [3] analysis methods often involve manual intervention for sample submission, monitoring, and interpretation of results. This manual process is time-consuming, prone to human error, and not scalable when handling a large number of malware samples generated daily in real-world scenarios.

Additionally, many available sandbox solutions provide extensive behavioral logs, but extracting meaningful indicators of malicious activity from these logs requires technical expertise and substantial analysis time. The absence of an automated mechanism for systematic behavior extraction and preliminary malicious activity identification creates a gap between malware execution and actionable threat intelligence.

Therefore, the core problem addressed in this project is the lack of a fully automated sandbox-based system capable of executing executable files, capturing relevant behavioral data, processing analysis reports, and assisting in identifying malicious behavior with minimal human intervention. Addressing this problem is essential to improve malware detection efficiency, reduce analysis overhead, and enhance the overall responsiveness of cybersecurity defense mechanisms.

1.3 Objectives of the Project

The primary objective of this project is to design and implement an automated sandbox-based system for detecting malicious behaviour in executable files. The system aims to execute suspicious files in an isolated environment, monitor their runtime activities, and analyze behavioral patterns to assist in identifying potential malware threats.

The specific objectives of the project are as follows:

- To study and understand existing malware analysis and detection techniques, including static and dynamic analysis methods.
- To design a controlled virtual environment for safe execution and analysis of suspicious executable files.
- To implement a sandbox framework capable of capturing system-level and network-level behavioral activities of executed samples.
- To develop automation mechanisms for sample submission, execution, and report generation within the sandbox environment.
- To collect and process behavioral logs to extract meaningful indicators of malicious activity.
- To evaluate the effectiveness of the proposed system in detecting malicious behavior based on observed execution results.

By achieving these objectives, the project intends to provide a practical and scalable solution that enhances malware analysis [12] efficiency and supports cybersecurity professionals in threat detection and investigation.

1.4 Scope of the Project

The scope of this project is focused on the development of an automated sandbox-based system for analyzing and detecting malicious behaviour in executable files. The system is designed to provide a controlled and isolated environment in which suspicious executable samples can be safely executed without affecting the host system. The project primarily emphasizes dynamic malware analysis by observing runtime behaviors rather than relying solely on static code inspection.

The proposed system covers the setup of a virtualized malware analysis lab, deployment of a sandbox framework, and automation of the sample execution process. It includes mechanisms

for monitoring system activities such as process creation, file system modifications, registry changes, and network communications triggered by the executed samples. Behavioral data collected during execution is processed to assist in identifying potential indicators of malicious activity.

This project is limited to the analysis of Windows-based executable files, as most common malware targets Windows operating systems. The detection approach is based on behavioral observation and report generation, rather than real-time blocking or removal of malware from infected systems. The system is intended for research, educational, and controlled laboratory use and does not function as a commercial antivirus [9] replacement.

Overall, the project aims to demonstrate the effectiveness of sandbox automation in malware behavior analysis and to provide a foundational framework that can be extended for advanced detection techniques in future enhancements.

1.5 Limitations

Although the proposed automated sandbox-based system provides an effective approach for analyzing and detecting malicious behaviour in executable files, certain limitations are associated with the scope and implementation of the project. One of the primary limitations is that the system relies on dynamic analysis, which requires the malware sample to execute successfully inside the sandbox. If a sample fails to execute or delays its malicious activity, the system may not capture complete behavioral data.

Some advanced malware variants are designed with sandbox-evasion and virtual machine detection techniques. Such malware may alter their behavior or remain dormant when executed in a virtualized environment, thereby reducing the accuracy of behavioral analysis. Additionally, the effectiveness of detection depends on the duration of sample execution; short execution windows may fail to reveal delayed malicious actions [7].

The project is limited to analyzing Windows-based executable files and does not currently support other file formats such as scripts, documents with embedded macros, or mobile application packages. Furthermore, the system focuses on behavioral data collection and preliminary identification of malicious activity but does not implement advanced machine learning-based classification or real-time threat blocking mechanisms.

Finally, the sandbox environment requires significant system resources, including memory, storage, and processing power, which may limit scalability when analyzing a large number of samples simultaneously. Despite these limitations, the system serves as a strong foundational framework for automated malware behavior analysis and can be extended in future work.

1.6 Organization

This report is organized into five CHAPTERs, each presenting different aspects of the project in a structured and systematic manner.

CHAPTER 1 introduces the project by presenting the background and motivation, defining the problem statement, stating the objectives, outlining the scope, discussing the limitations, and describing the organization of the report.

CHAPTER 2 provides a detailed review of malware and cybersecurity fundamentals, including types of malwares, attack vectors, and traditional detection techniques. It also discusses dynamic and sandbox-based analysis methods, existing sandbox frameworks, and research gaps that justify the need for an automated sandbox-based malware behaviour analysis system.

CHAPTER 3 explains the system design and methodology of the proposed solution. It describes the overall system architecture, malware analysis lab environment, sandbox automation workflow, sample execution process, behavioral data collection mechanisms, log processing methods, and rule-based detection logic used for identifying malicious activities.

CHAPTER 4 discusses the implementation and experimental analysis of the project. It covers the tools and technologies used, implementation of the Cuckoo Sandbox environment, automation script development, dataset preparation, experiment execution procedure, and observed behavioral patterns from analyzed samples.

CHAPTER 5 concludes the report by summarizing the project work, highlighting key contributions and findings, discussing system limitations, and suggesting possible future enhancements.

CHAPTER 2

LITERATURE REVIEW

2.1 Overview of Malware and Cyber Threat Landscape

The global cyber threat landscape has evolved rapidly with the expansion of digital infrastructure, cloud computing, mobile technologies, and the Internet of Things (IoT). As organizations increasingly rely on interconnected systems for data storage, communication, and business operations, cyber attackers have gained more opportunities to exploit vulnerabilities for malicious purposes. Among the various cyber threats, malware remains one of the most dominant and damaging attack vectors.

Malware, short for malicious software, refers to programs intentionally designed to infiltrate, damage, disrupt, or gain unauthorized access to computer systems. Over the years, malware has evolved from simple viruses causing minor disruptions to sophisticated threats capable of large-scale data breaches, financial fraud, espionage, and infrastructure sabotage. Modern malware often operates silently in the background, making detection challenging for conventional security tools.

The current cyber threat landscape includes a wide range of malware types such as viruses, worms, trojans, ransomware, spyware, adware, rootkits, and botnets. Attackers employ advanced techniques including polymorphism, metamorphism, encryption, and code obfuscation to evade signature-based detection. In addition, malware is frequently distributed through phishing emails, malicious websites, infected software downloads, removable media, and exploit kits targeting system vulnerabilities.

Cybercrime has also become highly organized and commercially driven. Ransomware-as-a-Service (RaaS), Malware-as-a-Service (MaaS), and underground marketplaces allow even low-skilled attackers to launch effective cyber-attacks. Nation-state actors and advanced persistent threats (APTs) further contribute to the complexity of the threat environment by developing highly targeted and stealthy malware for espionage and sabotage purposes.

These evolving threats demand advanced detection and analysis mechanisms capable of identifying previously unknown malware. As a result, behavioral and sandbox-based malware analysis techniques have gained importance in modern cybersecurity frameworks [10].

Understanding the overall malware and cyber threat landscape is essential for designing effective automated detection systems such as the one proposed in this project.

2.2 Types of Malware and Attack Vectors

Malware exists in various forms, each designed to perform specific malicious activities such as data theft, system disruption, unauthorized access, or financial exploitation. Understanding different types of malwares and their attack vectors is essential for developing effective detection and analysis systems. Modern malware is often engineered to evade traditional security measures and spread rapidly across networks and devices.

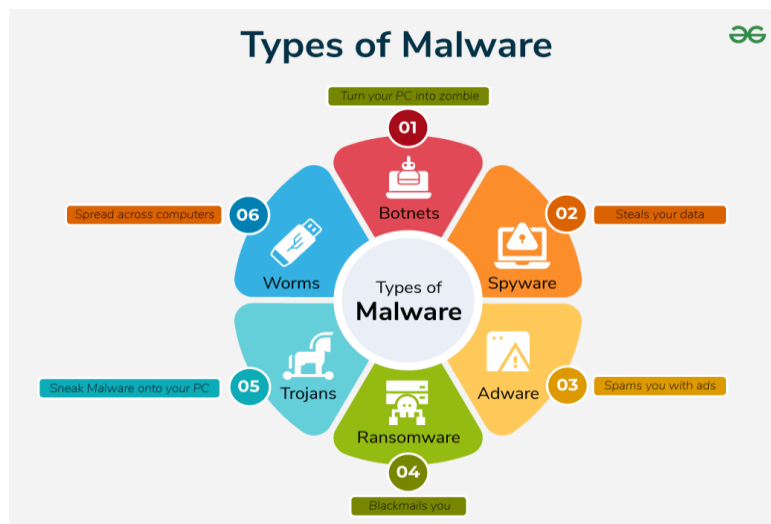


Figure 2.2.1 Classification of Common Types of Malwares

Botnets are networks of infected computers remotely controlled by attackers to launch large-scale attacks like DDoS or mass spam campaigns.

Spyware secretly monitors user activity and collects sensitive information such as passwords, keystrokes, and personal data.

Adware displays unwanted advertisements and may track browsing behavior to generate revenue for attackers.

Ransomware encrypts files or locks systems and demands payment to restore access to data [2].

Trojans disguise themselves as legitimate software but execute hidden malicious actions once installed, such as opening backdoors.

Worms self-replicate and spread automatically across systems and networks without user intervention, consuming resources and propagating rapidly.

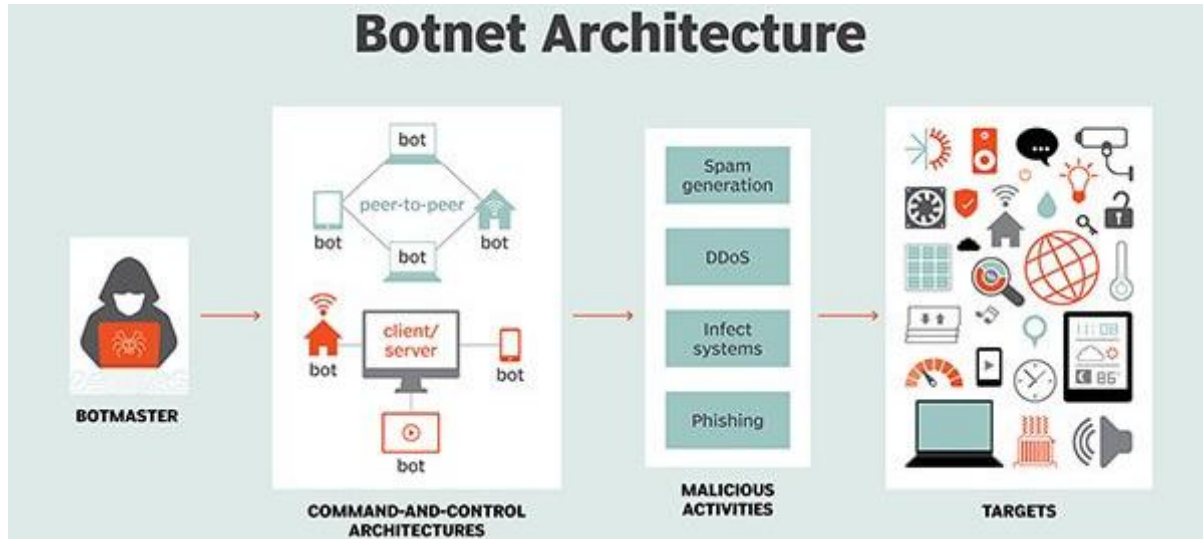


Figure 2.2.2 Botnet Command-and-Control Architecture

Malware reaches target systems through multiple attack vectors. Phishing emails containing malicious attachments or links remain one of the most common infection methods. Malicious websites and drive-by downloads exploit browser vulnerabilities to install malware without user consent. Infected software installers and cracked applications serve as another major distribution channel. Removable storage devices, such as USB drives, can spread malware across isolated systems. Additionally, exploiting unpatched system vulnerabilities allows attackers to install malware remotely.

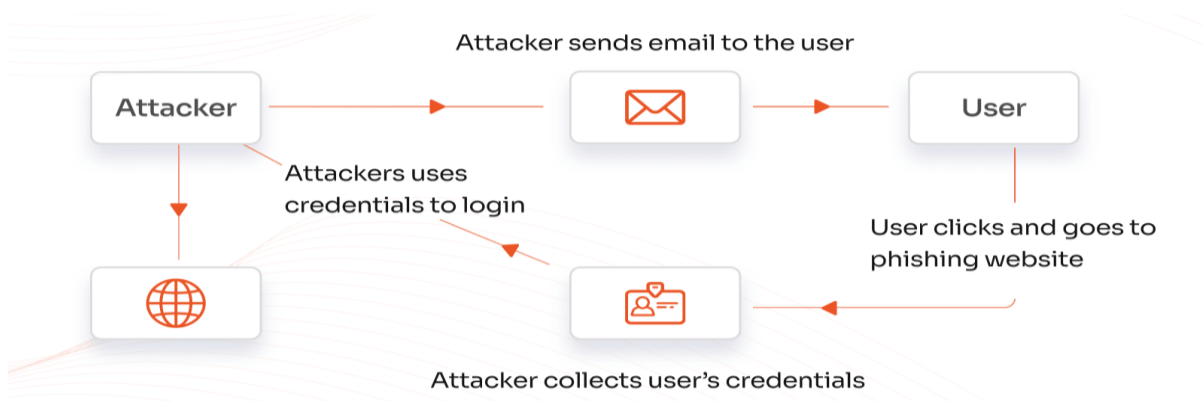


Figure 2.2.3 Workflow of a Phishing Attack

In recent years, social engineering techniques have been widely used to trick users into executing malicious files voluntarily. Furthermore, attackers employ fileless malware and living-off-the-land techniques, which use legitimate system tools to perform malicious actions, making detection even more difficult.

The diversity of malware types and attack vectors highlights the need for dynamic and automated analysis systems capable of identifying malicious behavior beyond traditional signature-based detection methods.

2.3 Traditional Malware Detection Techniques

Traditional malware detection techniques form the foundation of early cybersecurity defense systems. These methods were primarily developed to identify known malicious software based on predefined patterns, signatures, and rule-based inspections. Although modern threats have evolved significantly, traditional detection approaches remain important as baseline security mechanisms and are often combined with advanced analysis techniques [5].

Conventional detection techniques can be broadly categorized into static and behavior-based inspection methods. Static methods analyze executable files without running them, focusing on code patterns, signatures, and file characteristics. Behavior-based approaches examine program actions either through limited heuristic inspection or controlled execution environments. These techniques are widely used in commercial antivirus products due to their speed and low computational overhead.

However, traditional detection methods face significant challenges in identifying modern malware that uses obfuscation, encryption, polymorphism, and fileless execution techniques. As a result, these methods often fail to detect newly emerging or modified malware variants. Understanding the strengths and limitations of traditional techniques is essential before exploring dynamic and sandbox-based analysis systems.

Table 2.1 provides a comparison of commonly used traditional malware detection techniques.

Detection Technique	Analysis Type	Strengths	Limitations
Signature-Based Detection	Static	Fast and accurate for known malware	Cannot detect unknown or modified malware

Heuristic-Based Detection	Static / Limited Run	Detects some new malware variants	May produce false positives
Static Analysis	Static	Safe and resource efficient	Ineffective against obfuscated or packed malware
Basic Behavior Monitoring	Runtime	Observes suspicious activities	Limited visibility without full sandbox execution

2.3.1 Signature-Based Detection

Signature-based detection is one of the earliest and most widely used techniques for identifying malicious software. This approach relies on predefined patterns, known as signatures, which are extracted from previously identified malware samples. Antivirus and security tools compare files and system activities against a database of known signatures to determine whether a file is malicious.

A malware signature may consist of specific byte sequences, hash values, code fragments, or characteristic behavioral indicators unique to a particular malware family. When a new file is scanned, the detection engine searches for matches between the file's content and the stored signature database. If a match is found, the file is flagged as malicious and appropriate action is taken.

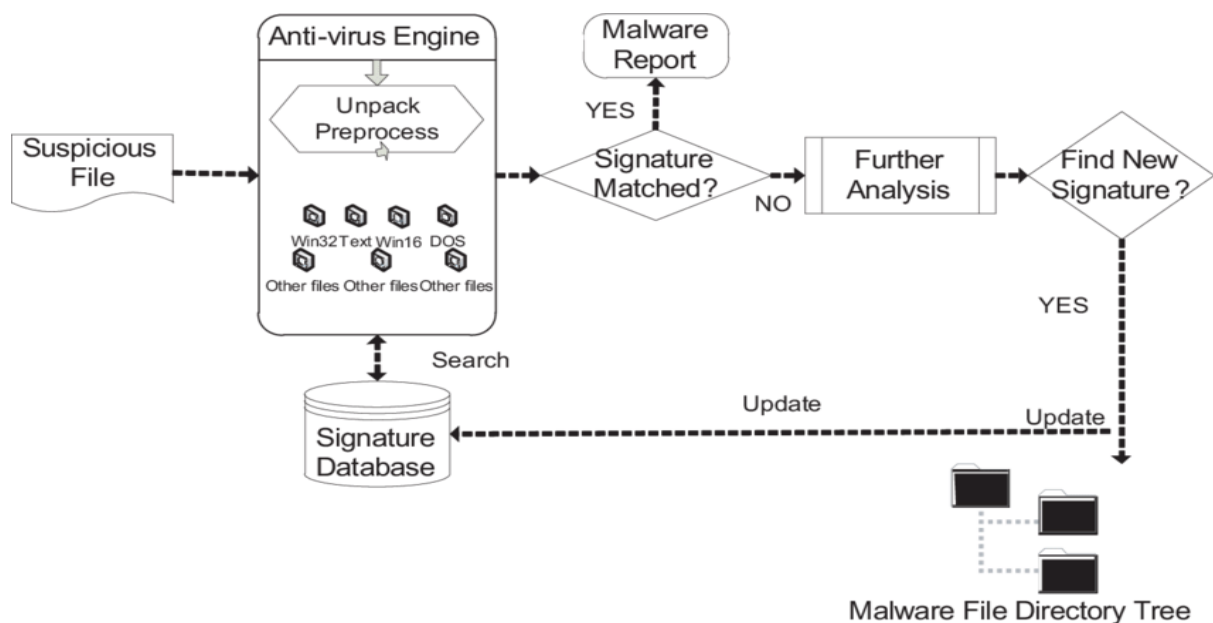


Figure 2.3.1 Signature-Based Malware Detection Process

The primary advantage of signature-based detection is its high accuracy in identifying known malware with minimal false positives. It is also computationally efficient and widely implemented in commercial antivirus solutions. However, this technique suffers from significant limitations when dealing with new, modified, or previously unseen malware. Attackers often employ polymorphism, encryption, and code obfuscation techniques to alter malware signatures [15], enabling them to bypass signature-based scanners.

Another major challenge is the continuous need to update signature databases. Security vendors must constantly collect new malware samples, analyze them, and generate new signatures. Any delay in updates can leave systems vulnerable to emerging threats.

Due to these limitations, signature-based detection alone is no longer sufficient to defend against modern cyber threats. This has led to the development and adoption of heuristic and behavior-based detection methods, which aim to identify malicious activities even when no known signature exists.

2.3.2 Heuristic-Based Detection

Heuristic-based detection is an advanced malware identification technique developed to overcome the limitations of traditional signature-based detection. Instead of relying solely on known malware signatures, this approach analyzes files and programs for suspicious characteristics or behaviors that may indicate malicious intent. Heuristic methods allow security systems to detect previously unknown or modified malware variants.

In heuristic detection, files are examined for abnormal code structures, unusual instruction sequences, suspicious API calls, and irregular execution patterns. These characteristics are evaluated against a set of predefined heuristic rules or behavioral indicators. If a file exhibits a combination of suspicious traits, it is flagged as potentially malicious, even if no known signature match exists.

This detection method is effective against polymorphic and obfuscated malware, which frequently change their code appearance to evade signature-based scanners. Heuristic analysis can be performed through static inspection of code or through limited execution in a controlled environment to observe suspicious behavior patterns.

However, heuristic-based detection has certain limitations. Because it relies on rule-based judgments and pattern recognition, it may produce false positives by incorrectly classifying

legitimate software as malicious [12]. Additionally, highly sophisticated malware can be designed to mimic normal program behavior, thereby reducing the effectiveness of heuristic detection.

Despite these challenges, heuristic-based detection remains a critical component of modern cybersecurity solutions. When combined with signature-based and behavioral analysis techniques, it enhances the overall capability of malware detection systems to identify both known and emerging threats.

2.3.3 Static Analysis Methods

Static analysis is a malware detection and examination technique in which executable files are analyzed without being executed. The primary objective of static analysis is to inspect the internal structure, code, and metadata of a file to identify suspicious patterns or indicators of malicious intent. This method is widely used as a preliminary step in malware analysis because it allows safe inspection without the risk of activating harmful behavior.

Static analysis involves examining file headers, import tables, embedded strings, hash values, and binary code sequences. Common static analysis techniques include hash-based identification, string analysis, opcode inspection, and disassembly. These methods help analysts identify known malware signatures, suspicious function calls, encrypted payloads, or hidden instructions. Tools such as disassemblers, debuggers, and file analysis utilities are commonly used to perform static inspection [6].

One of the key advantages of static analysis is that it is fast, resource-efficient, and does not require execution of the suspicious file. It also allows analysts to extract useful information such as embedded URLs, command-and-control addresses, and encryption keys without running the malware. Additionally, static analysis is effective in detecting known malware families through signature or hash matching.

However, static analysis has significant limitations when dealing with modern malware. Many advanced malware samples use packing, encryption, or code obfuscation techniques to hide their true behavior. In such cases, static inspection may fail to reveal meaningful information. Furthermore, static analysis cannot observe runtime behaviors such as file system changes, network communication, or process injection.

Due to these limitations, static analysis is often combined with dynamic and behavioral analysis techniques to provide a more comprehensive understanding of malware behavior.

2.4 Dynamic Malware Analysis

Dynamic malware analysis is a technique in which suspicious executable files are executed in a controlled and isolated environment to observe their behavior at runtime [1]. Unlike static analysis, which inspects code without execution, dynamic analysis focuses on monitoring real-time activities performed by a program when it runs. This approach is particularly effective for identifying modern malware that uses obfuscation or encryption to hide its malicious code.

During dynamic analysis, the malware sample is executed inside a secure environment such as a virtual machine or sandbox [6]. System activities including process creation, file system modifications, registry changes, memory usage, and network communications are continuously monitored and recorded. These observed behaviors provide valuable insight into the actual intent and impact of the malware.

One of the major advantages of dynamic analysis is its ability to detect previously unknown malware based on behavioral patterns rather than known signatures. It also allows analysts to uncover hidden payloads, dropped files, and command-and-control communication that may not be visible through static inspection.

However, dynamic analysis also faces challenges. Some advanced malware variants can detect virtualized or sandbox environments and intentionally alter their behavior to avoid detection. Additionally, if malware is designed to execute its malicious actions after a long delay or under specific conditions, short analysis time windows may fail to capture complete behavior.

Despite these limitations, dynamic malware analysis remains a crucial component of modern cybersecurity research and threat detection. It forms the foundation for sandbox-based automated malware analysis systems, such as the solution proposed in this project.

2.5 Sandbox-Based Analysis Techniques

Sandbox-based analysis is an advanced form of dynamic malware analysis in which suspicious files are executed inside an isolated and controlled environment [1] known as a sandbox. A sandbox is typically implemented using virtualization or container-based technologies to create

a secure environment that mimics a real operating system. This allows malware to run freely while preventing any harm to the host system or external networks.

In sandbox-based analysis, the submitted executable file is launched within a virtual machine configured with monitoring tools. During execution, the sandbox continuously observes and logs system-level and network-level activities such as process creation, file operations, registry modifications, API calls, and outbound network connections. These collected logs form a detailed behavioral report that helps in understanding the actions performed by the suspicious file.

One of the key benefits of sandbox-based analysis is automation. Modern sandbox frameworks can automatically execute samples, monitor behavior, generate reports, and reset the environment for subsequent analyses [8]. This automation significantly reduces the manual effort required in malware investigation and allows large-scale sample processing.

Sandbox-based techniques are highly effective in detecting unknown and zero-day malware threats since detection is based on runtime behavior rather than static signatures. Additionally, these techniques provide valuable forensic evidence that can be used for threat intelligence and incident response.

However, sophisticated malware may incorporate sandbox detection and evasion techniques to avoid revealing malicious behavior. Moreover, sandbox environments require careful configuration to closely resemble real user systems; otherwise, malware may not execute its full payload. Despite these challenges, sandbox-based analysis remains a powerful and widely adopted approach in modern malware detection and research.

2.6 Existing Sandbox Frameworks

With the growing demand for automated malware analysis, several sandbox frameworks have been developed to provide dynamic analysis capabilities. These frameworks allow security researchers and organizations to safely execute suspicious files, observe their behavior, and generate detailed analysis reports. Existing sandbox solutions vary in architecture, features, scalability, and level of automation, but all aim to enhance malware detection through behavioral monitoring.

Cuckoo Sandbox is one of the most widely used open-source sandbox frameworks for malware analysis [1]. It supports automated sample submission, virtual machine management,

behavior monitoring, and report generation. Cuckoo provides detailed logs of file system activities, registry changes, process execution, and network traffic, making it highly suitable for research and academic projects. Its modular architecture allows customization and integration with additional analysis tools.

Any.Run is a popular interactive online sandbox platform that provides real-time malware execution and behavior visualization. It allows analysts to manually interact with running malware samples, making it useful for in-depth behavioral investigations. However, as a cloud-based commercial service, it has usage limitations and is not suitable for offline lab-based deployment.

Hybrid Analysis is another well-known sandbox solution that combines static and dynamic analysis techniques. It offers automated report generation and integrates multiple detection engines to provide comprehensive threat assessment. Like Any.Run, it operates primarily as an online service, which may limit customization and control over the analysis environment.

In addition to these, enterprise-level sandbox systems such as Joe Sandbox and FireEye Sandbox offer advanced threat detection capabilities [4] for commercial use. However, these solutions are often expensive and proprietary, making them less accessible for academic and small-scale research environments.

The existence of these frameworks demonstrates the effectiveness of sandbox-based malware analysis. However, many available solutions either require paid subscriptions, lack full automation customization, or operate as closed platforms. This creates a need for an open, customizable, and automated sandbox-based system, which forms the basis of the proposed project.

2.6.1 Cuckoo Sandbox

Cuckoo Sandbox is an open-source automated malware analysis framework widely used in cybersecurity research and academic environments. It provides a controlled and isolated environment where suspicious executable files can be safely executed and monitored to observe their runtime behavior. Due to its flexibility, extensibility, and free availability, Cuckoo Sandbox has become one of the most popular sandbox solutions for dynamic malware analysis.

Cuckoo operates by running malware samples inside virtual machines configured with monitoring agents. When a sample is submitted, the framework automatically launches a clean virtual machine, executes the file, monitors system activities, records behavioral events, and then restores the virtual machine to its original state. The collected data is processed to generate detailed analysis reports containing information about process execution, file system changes, registry modifications, API calls, memory usage, and network communications.

One of the key strengths of Cuckoo Sandbox is its modular architecture. It allows integration of additional tools such as network traffic analyzers, memory dump utilities, and third-party antivirus scanners. Users can customize analysis parameters, execution time, virtual machine configurations, and reporting formats based on specific project requirements. This makes Cuckoo highly suitable for building automated malware analysis laboratories.

Despite its advantages, Cuckoo Sandbox also has limitations. Advanced malware samples may detect the virtualized environment and suppress malicious behavior to evade detection [14]. Additionally, setting up and maintaining the Cuckoo environment requires technical expertise in virtualization, networking, and system configuration. However, with proper setup and tuning, Cuckoo Sandbox remains a powerful platform for automated behavioral malware analysis.

In this project, Cuckoo Sandbox is selected as the core analysis engine due to its open-source nature, automation capabilities, and compatibility with customized malware analysis workflows.

2.6.2 Any.Run Sandbox

Any.Run is a cloud-based interactive malware analysis platform designed to provide real-time dynamic analysis of suspicious files. Unlike traditional automated sandboxes, Any.Run allows analysts to interact directly with the running sample through a remote virtual environment. This interactivity enables deeper investigation of malware behavior that may require user actions such as clicking buttons, entering inputs, or opening embedded files.

The platform executes submitted samples inside cloud-hosted virtual machines and provides live monitoring of system processes, file system activities, registry changes, and network communications. Analysts can observe malware behavior as it unfolds and control the execution environment in real time. Any.Run also generates detailed behavioral reports and threat assessments after analysis is completed.

One of the major advantages of Any.Run is its ease of use and accessibility. Since it operates entirely online, it does not require local installation or complex environment setup. It also provides visual representations of behavioral events, making it easier for security researchers to understand malware activity patterns [15].

However, Any.Run is a commercial service with limited free usage and restricted advanced features behind paid subscriptions. Additionally, because it is cloud-based, users have limited control over low-level system configuration and environment customization. It may also not be suitable for analyzing highly sensitive or confidential malware samples due to data privacy considerations.

Despite these limitations, Any.Run remains a valuable sandbox platform for quick interactive malware investigation and behavioral visualization, especially for demonstration and preliminary analysis purposes.

2.6.3 Hybrid Analysis

Hybrid Analysis is an online malware analysis platform that combines both static and dynamic analysis techniques to provide comprehensive insight into suspicious files. It is widely used by security researchers and incident response teams for rapid malware investigation and threat assessment. The platform allows users to submit executable files, documents, and URLs for automated analysis in a cloud-based sandbox environment.

In Hybrid Analysis, submitted samples undergo initial static inspection to extract metadata, file hashes, embedded strings, and structural characteristics. Following static analysis, the samples are executed in an isolated virtual environment where their runtime behavior is monitored. The platform records system changes, process activities, network communications, and dropped files, generating a detailed report that includes both static and dynamic findings.

One of the key strengths of Hybrid Analysis is its multi-engine scanning capability. It integrates multiple antivirus and threat detection engines to provide comparative detection results alongside behavioral reports. This combined approach improves detection accuracy and helps analysts correlate behavioral patterns with known threat classifications.

Hybrid Analysis offers an intuitive web interface and does not require local sandbox setup, making it easily accessible. However, like other cloud-based services, it has limitations in customization and environment control. Advanced features and extended analysis options are

available only through premium access. Additionally, sensitive malware samples may raise confidentiality concerns when uploaded to third-party servers.

Despite these constraints, Hybrid Analysis remains a powerful tool for quick and comprehensive malware assessment and is widely recognized in the cybersecurity community for its hybrid static-dynamic [2] analysis approach.

2.7 Research Gaps and Need for Automation

The rapid evolution of malware and the increasing complexity of cyber threats have exposed several limitations in existing malware detection and analysis techniques. While static and dynamic analysis methods, along with sandbox-based frameworks, have significantly improved threat detection capabilities, multiple research gaps still remain [4] in achieving fast, scalable, and fully automated malware behavior identification.

Most traditional detection systems rely heavily on signature-based or rule-based approaches, which struggle to identify zero-day malware and continuously evolving malicious variants.

Although sandbox-based dynamic analysis

provides detailed behavioral insights, many existing frameworks require manual intervention for sample submission, execution monitoring, and interpretation of reports. This manual dependency increases analysis time and limits scalability when handling large volumes of malware samples generated daily.

Another significant research gap lies in automated behavioral interpretation. Current sandbox solutions generate extensive logs and reports, but extracting meaningful indicators of compromise from these reports often requires expert knowledge. There is limited integration of automated decision-making mechanisms that can assist in preliminary classification of malicious behavior based on observed activity patterns.

Additionally, advanced malware increasingly incorporates sandbox-evasion techniques and delayed execution strategies [3][7][12][14], which can reduce the effectiveness of conventional sandbox environments. This highlights the need for improved sandbox automation, environment realism, and adaptive analysis workflows capable of triggering malware behavior more effectively.

Resource efficiency and scalability also remain challenges. Many existing sandbox solutions require high computational resources and complex configurations, making large-scale deployment difficult in academic or small organizational environments.

These gaps collectively emphasize the need for an automated sandbox-based system that can handle sample execution, behavioral data collection, log processing, and preliminary malicious behavior identification with minimal human involvement.

Table 2.7: Comparative Analysis of Existing Malware Analysis and Detection Techniques

Ref No	Paper Title	Methodology	Tools Used	Algorithm	Dataset	Acc/Res	Limitations
[1]	Going beyond API Calls in Dynamic Malware Analysis - A Novel Dataset	Large-scale Cuckoo executions; extract full sandbox report fields (API, file/registry/network/process/memory events) and compare full-featured vs API-only feature sets; train tree ensembles with 10-fold CV.	Cuckoo Sandbox, custom Python extractor, scikit-learn/WEKA.	Random Forest (best), Gradient Boosting, Gaussian NB, Neural Networks.	22,200 Cuckoo reports (multi-file types; heavy feature set).	≈99%	Enormous compute/memory needs; Windows PE focus; possible VM-evasion by sophisticated malware.
[2]	Dynamic Feature Dataset for Ransomware Detection	Execute ransomware samples across Windows variants in Cuckoo, extract 326 dynamic features, apply feature selection to 50 features, and evaluate ML models with 10-fold CV.	Cuckoo Sandbox, Python feature parser, scikit-learn, TensorFlow.	Random Forest (top), Gradient Boosted Regression Trees, Neural Networks, SVM.	Ransomware-focused dataset; cross-Windows testing (multiple OS versions).	>99%	Focus on known ransomware families—may not generalize to novel/unknown families; Windows-only; feature selection risks omitting new behaviors.
[3]	Investigating Anti-Evasion Malware Triggers	Automated + manual sandbox reconfiguration experiments (VirtualBox/Cuckoo) to trigger anti-evasion behaviors — vary guest-additions, system artifacts, time jumps, simulated user activity — and observe detection differences.	MORRIGU system, VirtualBox, Cuckoo Sandbox, Windows test images.	Behavioral analyses; Random Forest used for detection comparison and trigger influence.	Smaller, curated sample set (≈251 samples) focusing on evasive binaries.	NS	Small dataset; manual tuning and resource-intensive experimentation ; cannot exhaust all evasion techniques.

LITERATURE REVIEW

[4]	Tools and Techniques for Malware Detection and Analysis	Literature survey and taxonomy across static, dynamic, and hybrid analysis tools and techniques; compares tool capabilities and typical workflows.	(Surveyed) Cuckoo, DRAKVUF, Joe Sandbox, Volatility, IDA Pro, Wireshark, YARA, etc.	Overview of signature-based methods, anomaly detection, various ML classifiers, YARA rules, opcode heuristics.	N/A(survey)	NS	No original experiments or quantitative metrics; quickly dated as tools evolve.
[5]	Attention-Based Automated Feature Extraction for Malware Analysis	Extract API-call sequences (up to ~800 calls) and apply a Skip-Connected LSTM with attention to highlight informative calls; compare against CNN and SC-LSTM baselines; interpret top-k subsequences.	Cuckoo Sandbox, Python, Theano, custom LSTM+attention code.	Attention-based LSTM (primary), SC-LSTM baseline, CNN for comparison.	API-sequence datasets from sandboxed executions (PE binaries).	NS	High training time, sequence-length constraints, dependent on reliable API extraction, Windows/PE focus.
[6]	ZeVigilant — Detecting Zero-Day Malware	Hybrid static + dynamic analysis; extract PE imports and API sequences; apply multiple ML models with cost-sensitive/balancing techniques and cross-validation.	Cuckoo Sandbox, WEKA, Python, VirusTotal for labeling.	Random Forest (best), Neural Networks, Decision Trees, k-NN, SVM.	Large dataset (~40k+ samples) spanning malware families and goodware.	≈98%	Dataset imbalance concerns; heavy dependence on Cuckoo; Windows-only focus.
[7]	How Long Does It Take for the Malware to Activate	Large temporal study using PANDA system-call traces; minute-by-minute execution windows with RNN/LSTM and tree models to evaluate how detection accuracy varies with execution time.	PANDA, system-call tracers, sequential mining toolchain, RNN/LSTM toolkits.	RNN, LSTM, Gradient-Boosted Trees and cascaded tree models.	Very large sample set (tens of thousands; reported up to 100k in related notes).	NS	Time-bomb/sleep evasions can still evade; platform specifics; multi-stage malware complexities.
[8]	Collaborative, Lightweight Malware Analysis Sandbox (SecBox)	Build container-based sandbox using gVisor and seccomp for isolation; provide web UI and multi-user collaboration; compare performance & scalability versus VM sandboxes.	Docker, gVisor, seccomp, Python/Django backend, web frontend, system-call capture.	Behavioral anomaly detection and pattern matching (RF used for validation).	Case studies (Mirai, coinminer, etc.) and internal test workloads.	NS	Shared kernel — weaker isolation than VMs; Linux-focused; limited visibility for kernel/rootkit threats.
[9]	A User-friendly Model for Ransomware Analysis	Hybrid static + dynamic analysis wrapped in a user-friendly web UI that translates technical reports into simpler summaries; evaluated via usability surveys.	Cuckoo Sandbox, Python backend, HTML/CSS frontend, REST APIs.	Behavioral analysis and pattern matching; risk scoring heuristics.	Ransomware samples + user study participants for usability feedback	~90%	Lower detection accuracy vs advanced research systems; Windows-only; limited algorithmic novelty.
[10]	An Automated Tool for Malware Analysis and Classification	Hybrid pipeline combining hashing → YARA/PE static checks → dynamic API/registry logs → instruction-frequency extraction (objdump) → pattern-based classification.	YARA, PEiD, objdump, Process Monitor, custom DLL injection, Java + Python implementations.	Instruction Frequency Classifier and hybrid heuristics	Limited validation (Conficker and a few samples).	NS	Very small evaluation set; simplistic classifier; manual tuning required; Windows-only.

[11]	Dynamic Analysis of Window-Based Malware Using Automated Sandboxing	Combined static and dynamic malware analysis of the LockerGoga ransomware using an isolated virtual lab environment. Static analysis performed first, followed by dynamic execution inside Cuckoo sandbox with monitored system and network activity.	Static Analysis Tools: BinText, TrID, UPX, XORSearch, ExeinfoPE. Dynamic Analysis Tools: FakeNet, Procmon, ProcDOT, Wireshark, Process Explorer, RegShot, Cuckoo Sandbox.	RSA-2048 encryption used by LockerGoga ransomware. Also entropy analysis for packed malware detection and function call tracing (CreateFileA, ReadFile, etc.).	LockerGoga malware sample (Windows ransomware). No large dataset; single real-world malware sample analyzed.	Malware identified as packed (entropy = 6.62). CPU usage increased from 0% to 21% after execution. Registry modifications and DLL dependencies identified. No classification accuracy since this is analysis-based, not ML-based.	Heuristic-based tools are limited because they rely on repeated code patterns. The study suggests further research using advanced tools and methods is needed for better malware mitigation.
[12]	Malware Dynamic Analysis Evasion Techniques: A Survey	Comprehensive survey and classification of malware dynamic analysis evasion techniques for manual (anti-debugging) and automated (sandbox) analysis. Proposes taxonomy: detection-dependent and detection-independent evasion, with comparative evaluation.	Analysis based on debuggers, virtual machines, emulators, sandboxes, and bare-metal test environments discussed throughout the survey. No experimental tool implementation; study is literature-based.	No specific detection algorithm.	Reviewed academic and industry malware samples and reports; no fixed experimental dataset. Focus on Windows malware prevalence.	Identified fingerprinting as most pervasive sandbox evasion tactic; detection-independent tactics (stalling, fileless) emerging as major threats. No accuracy metrics since it is a survey paper.	Defensive methods remain largely reactive; zero-day fingerprinting and detection-independent tactics still evade current sandboxes. Path-exploration suggested as future solution.

[13]	Automated Malware Analysis Platform	Design and implementation of an automated in-house malware analysis infrastructure combining static and dynamic scanning. Built an integrated platform using IRMA as central framework and Cuckoo for dynamic analysis.	IRMA framework, Cuckoo Sandbox, multiple antivirus engines, Ansible, Vagrant, Packer, virtualization tools.	No specific detection algorithm proposed; system relies on signature-based, heuristic-based, and dynamic behavioral analysis through integrated tools.	Tested using benign and malicious files uploaded to the platform for scan evaluation.	Successful automated scanning pipeline; reported detection rates from integrated antivirus and Cuckoo scoring system. No ML accuracy metrics since it is an infrastructure-based system.	Scalability and advanced evasion-resistant detection were outside project scope; future work suggested for improving scalability and reporting.
[14]	Investigating Anti-Evasion Malware Triggers Using Automated Sandbox Reconfiguration Techniques	Proposed MORRIGU, a dynamic sandbox reconfiguration system combining automated and human-driven analysis to systematically test anti-evasion techniques and trigger evasive malware behaviours.	MORRIGU framework, VirtualBox, Windows 10 VM, PowerShell & VBS scripts, DNS VM, visual analytics dashboard.	No single detection algorithm; uses anti-evasion techniques: Guest Additions removal, Time Jump, Registry Edit, Reverse Turing Test, Common Artefact Creation to trigger malware.	251 real-world Windows malware samples from VirusShare, across 49 families and 10 classifications.	173/251 samples active; 40 samples (16%) were evasive and only triggered under anti-evasion configurations. Demonstrated failure of traditional sandboxes to detect such samples.	Cannot test all possible environment configurations; some malware may still remain dormant. Static analysis and more advanced environment modelling suggested as future work.
[15]	Attention-Based Automated Feature Extraction for Malware Analysis	Proposed an attention-based feature extraction method to identify important API system calls for malware detection. Combined static (opcode trigram) and dynamic (API call trigram) analysis, then trained deep learning models using attention-weighted subsequences.	Cuckoo Sandbox (API call extraction), Objdump (opcode extraction), Theano, Keras, GPU (GTX 1060).	Attention-based LSTM model with Skip-Connected LSTM (SC-LSTM) classifier. Uses attention weights to select top-k API subsequences.	PE files from HAURI dataset: 1000 benign + 1000 malicious files; final usable set: 690 benign + 785 malware samples.	Attention model achieved ~96.27%, about 12% higher than CNN and 5% higher than SC-LSTM.	Longer training time due to attention weight computation; relies on successful sandbox execution, so highly evasive malware may still avoid analysis.

CHAPTER 3

SYSTEM DESIGN AND METHODOLOGY

3.1 Proposed System Overview

The proposed system is an automated sandbox-based framework designed to analyze executable files and detect malicious behaviour through dynamic execution and behavioral monitoring. The system provides a secure and isolated environment where suspicious files can be executed without risking the integrity of the host machine or connected networks. By automating sample execution and behavioral data collection, the system aims to improve the efficiency and consistency of malware analysis.

The core of the proposed system is built around a sandbox environment deployed using virtual machine technology. When a suspicious executable file is submitted, the system automatically launches a clean virtual machine instance, executes the file, and monitors its runtime activities. System-level events such as process creation, file modifications, registry changes, memory usage, and network communications are recorded during execution. After the analysis is completed, the virtual machine is restored to its original clean state to ensure a fresh environment for the next sample.

Behavioral logs collected during execution are processed to extract relevant indicators of malicious activity. These indicators assist in identifying whether the executed file demonstrates suspicious or harmful behavior. The system is designed to operate with minimal manual intervention, allowing multiple samples to be analyzed in a structured and repeatable manner.

The proposed framework integrates sandbox automation, behavioral monitoring, and report generation into a single workflow. This automation reduces the time required for malware investigation and provides consistent analysis results. The system is primarily intended for academic research and controlled laboratory environments, serving as a foundation for further enhancements such as machine learning-based classification and advanced threat intelligence integration.

Overall, the proposed system offers a practical and scalable approach to automated malware behaviour detection, addressing key limitations of traditional manual analysis methods.

3.2 System Architecture

The system architecture of the proposed automated sandbox-based malware analysis framework is designed to provide a structured, secure, and fully automated workflow for analyzing suspicious executable files. The architecture integrates sample submission, sandbox execution, behavioral monitoring, data collection, and report generation into a unified system. Each component plays a specific role in ensuring safe execution and accurate detection of malicious behavior.

The architecture begins with the Sample Submission Module, where suspicious executable files are provided as input to the system. Once a file is submitted, the Automation Controller initiates the analysis process by interacting with the sandbox environment. This controller is responsible for starting the virtual machine, transferring the sample to the sandbox, triggering execution, and managing the analysis duration.

The Sandbox Environment consists of a virtual machine configured with a guest operating system and monitoring agents. The suspicious file is executed inside this isolated environment. During execution, the Behavior Monitoring Module continuously tracks system activities such as process creation, file system changes, registry modifications, API calls, and network traffic. These events are captured in real time to ensure comprehensive behavioral observation.

Captured behavioral data is forwarded to the Log Collection and Processing Module, where raw logs are filtered and structured into meaningful analysis reports. The Detection Logic Module then evaluates the processed behavioral indicators to assist in identifying whether the executed file demonstrates malicious characteristics.

After the analysis is completed, the Report Generation Module produces a detailed output summarizing observed behaviors and potential threats. Finally, the virtual machine is reverted to a clean snapshot state, ensuring that each new analysis begins in a fresh and uncompromised environment.

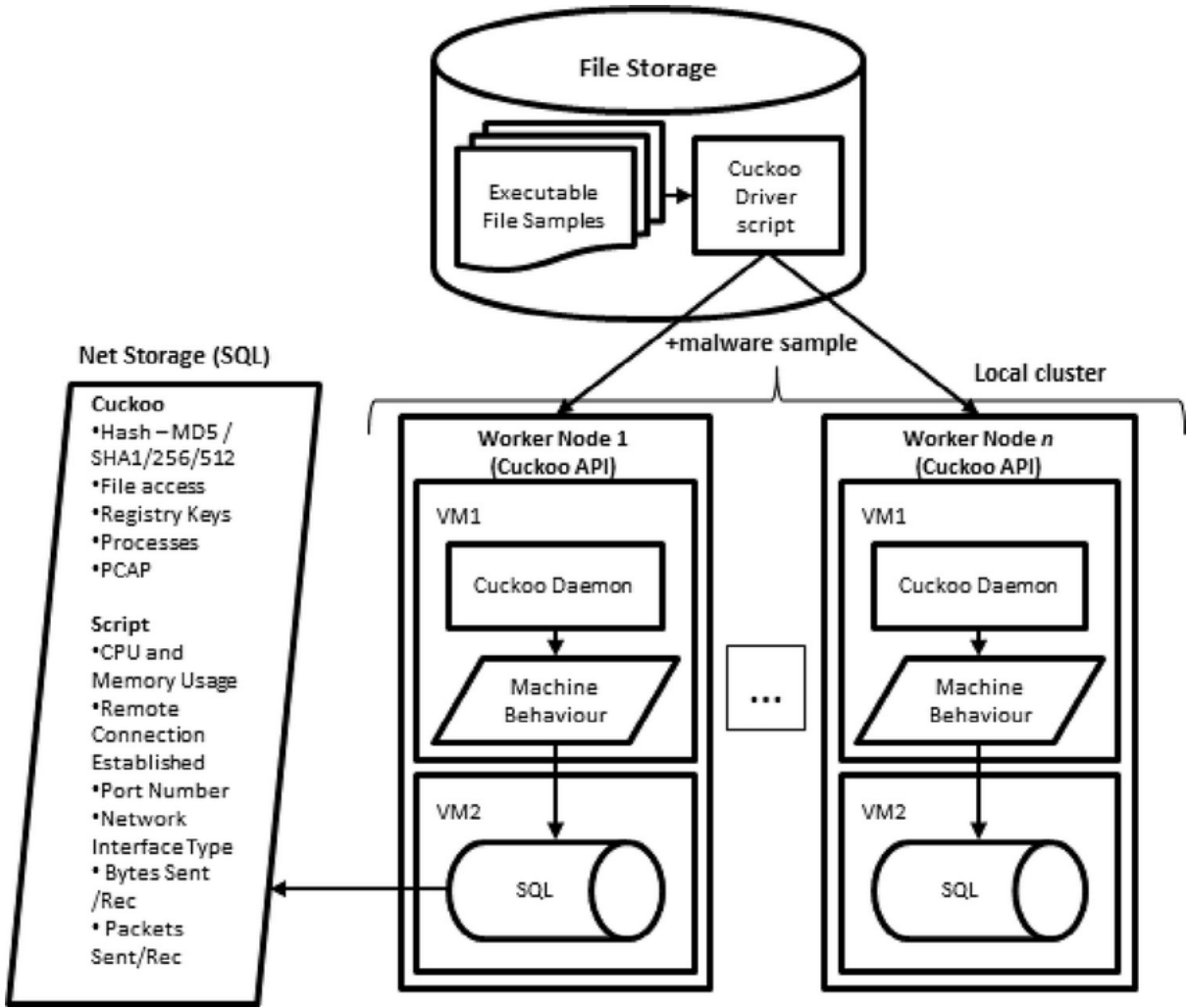


Figure 3.2.1 Distributed Cuckoo Sandbox Deployment Architecture

This modular architecture ensures automation, repeatability, and safety throughout the malware analysis process. It also allows future integration of advanced detection techniques such as machine learning-based classification and threat intelligence correlation, making the system scalable and adaptable to evolving cybersecurity requirements.

Table 3.2 Components of the Proposed Automated Sandbox-Based Malware Analysis System

Component	Description
Host System	Controls sandbox and automation
Guest VM	Executes suspicious files
Automation Controller	Manages execution workflow
Monitoring Agent	Collects behavioral data
Log Processor	Extracts behavior features

3.3 Malware Analysis Lab Environment Design

A properly designed malware analysis lab environment is essential to ensure safe execution of malicious samples and accurate collection of behavioral data. The proposed system uses a controlled virtualized lab setup that isolates malware execution from the host system and external networks. This environment provides a realistic operating system for malware to run while preventing any unintended spread or damage.

The lab environment consists of a host machine, a virtualization platform, and one or more guest virtual machines configured as sandbox instances. The host machine runs the main automation controller and manages virtual machine operations. A virtualization platform such as VirtualBox or VMware is used to create and manage the guest virtual machines. Each guest machine is installed with a Windows operating system, monitoring agents, and necessary tools required for sandbox execution.

To ensure repeatable and clean analysis, a baseline snapshot of the guest virtual machine is created after installation and configuration. Before analyzing a new sample, the system reverts the virtual machine to this clean snapshot state. This guarantees that each malware sample is executed in an identical and uncompromised environment.

The network configuration of the lab is designed to control malware communication. A host-only or isolated network is used to allow monitoring of network activity without exposing external systems to potential attacks. Optional simulated internet services or fake DNS and HTTP servers may be included to observe command-and-control communication attempts by malware.

Monitoring components installed in the guest machine capture runtime activities such as file operations, registry changes, process behavior, and network traffic. These logs are transmitted

to the host system for storage and processing. The automation controller manages the complete lifecycle of analysis, including virtual machine startup, sample execution, monitoring, report collection, and snapshot restoration.

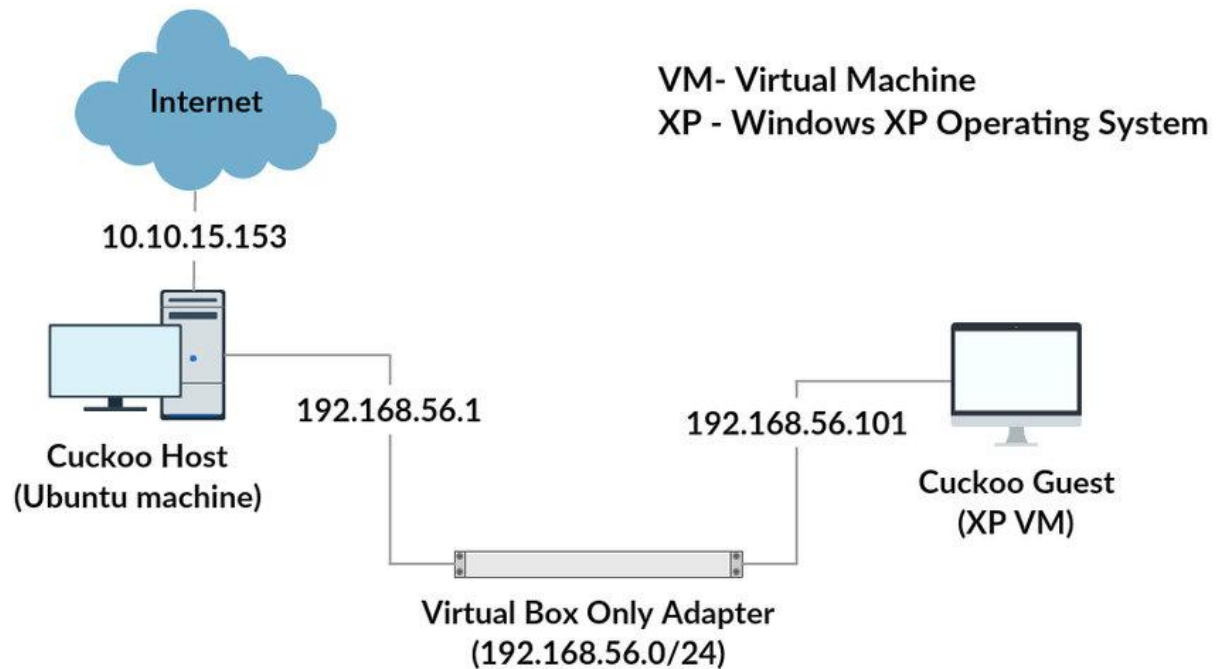


Figure 3.3.1 Network Configuration of Host and Guest Virtual Machines in Cuckoo Sandbox Environment

This lab environment design ensures safety, repeatability, and automation in malware behavior analysis. It forms the technical foundation of the proposed sandbox-based detection system and allows controlled experimentation with real malware samples in a secure research setting.

3.3.1 Host System Configuration

The host system serves as the central control unit of the malware analysis lab environment. It is responsible for managing the virtualization platform, coordinating sandbox automation, storing behavioral logs, and generating analysis reports. Proper configuration of the host system is essential to ensure stable operation of the sandbox environment and reliable execution of malware analysis tasks.

The host machine is equipped with sufficient hardware resources to support multiple virtual machines and continuous monitoring processes. A multi-core processor, adequate RAM, and high storage capacity are required to handle virtual machine execution, snapshot management, and log storage efficiently. The host operating system is typically a Linux-based distribution due to its stability, security, and compatibility with sandbox automation tools.

The virtualization platform installed on the host system enables the creation and management of guest virtual machines. Virtualization software such as VirtualBox or VMware is configured to allow snapshot creation, network isolation, and controlled resource allocation to guest systems. In addition, automation scripts and sandbox management tools are installed on the host to control virtual machine startup, sample submission, execution timing, and restoration processes.

The host system also includes log collection utilities and databases to store behavioral reports generated during malware execution. Network monitoring tools may be deployed to capture traffic between the guest machine and simulated network services. Access controls and firewall rules are configured to prevent malware from communicating with external networks or affecting the host environment.

Overall, the host system configuration ensures centralized control, secure isolation, and smooth automation of the malware analysis workflow, forming a stable backbone for the proposed sandbox-based detection framework.

3.3.2 Virtual Machine Setup

The virtual machine setup forms the core execution environment of the sandbox-based malware analysis system. It provides a controlled and isolated guest operating system where suspicious executable files can be safely run and monitored without risking the host system or external networks. Proper configuration of the virtual machine is essential to ensure realistic malware execution and accurate behavioral data collection.

A Windows operating system is installed on the guest virtual machine, as most common malware targets Windows platforms. After installation, essential system updates are applied, and unnecessary services are disabled to reduce background noise during behavioral monitoring. Monitoring agents and analysis tools are then installed to track system activities such as process creation, file modifications, registry changes, and network communication.

To maintain consistency across multiple analyses, a clean baseline snapshot of the virtual machine is created after the operating system and monitoring tools are configured. Before executing each new malware sample, the system automatically reverts the virtual machine to this snapshot. This ensures that every sample is analyzed in an identical and uncontaminated environment, improving the reliability of behavioral comparisons.

The virtual machine is allocated appropriate system resources, including CPU cores, memory, and storage space, to ensure smooth execution of malware samples. Resource allocation is carefully balanced to provide realistic performance while avoiding excessive load on the host system. Shared folders and clipboard synchronization features are disabled to prevent accidental data leakage between host and guest systems.

Additionally, the virtual machine is configured with controlled network settings to monitor malware communication while preventing external infection. Depending on analysis requirements, the network may be set to host-only mode or an isolated internal network with simulated internet services.

This virtual machine setup provides a secure, repeatable, and realistic environment for executing suspicious files, making it a critical component of the proposed automated sandbox-based malware analysis framework.

3.3.3 Network Configuration

Network configuration plays a crucial role in malware analysis, as many modern malware samples attempt to communicate with external servers for command-and-control operations, data exfiltration, or payload downloads. The network environment in the proposed sandbox system is carefully designed to observe such communications while preventing any real-world damage or external spread of malware.

The virtual machine used for malware execution is configured with an isolated network setup. A host-only or internal network mode is typically employed to ensure that the guest system can communicate with the host for monitoring and log transfer purposes, but has no direct access to the external internet. This prevents malware from contacting real command-and-control servers or spreading to external systems.

To simulate realistic network behavior, optional fake network services such as simulated DNS, HTTP, or proxy servers can be deployed on the host system. These simulated services allow the malware to attempt outbound connections, enabling observation of communication patterns without exposing actual networks. Network monitoring tools are installed on the host to capture and record traffic generated by the malware during execution.

Firewall rules and access control policies are enforced on the host system to block unauthorized outbound or inbound connections. This ensures that even if the malware attempts to bypass network restrictions, it cannot reach external systems. Network traffic logs collected during

analysis provide valuable indicators such as suspicious domains, IP addresses, communication protocols, and data transmission attempts.

This controlled network configuration enables safe observation of malware communication behavior while maintaining complete isolation from real-world networks. It enhances the accuracy of behavioral analysis and strengthens the security of the malware analysis lab environment.

3.4 Sandbox Automation Workflow

The sandbox automation workflow defines the sequence of operations performed by the proposed system to analyze suspicious executable files without manual intervention. Automation is a key feature of the proposed framework, as it ensures consistency, repeatability, and efficiency in malware analysis.

The workflow begins when a suspicious executable file is submitted to the system. The automation controller first verifies the input sample and stores it in a designated analysis directory. It then triggers the virtualization platform to launch a guest virtual machine from a clean baseline snapshot. Once the virtual machine is fully initialized, the automation controller transfers the sample into the guest environment and initiates its execution.

During sample execution, monitoring agents inside the virtual machine record system activities such as process creation, file system modifications, registry operations, API calls, and network communications. These events are continuously logged throughout the analysis duration. The automation controller maintains execution timing and ensures that the sample runs for a predefined observation period.

After the execution phase is completed, the monitoring logs are collected from the guest machine and transferred to the host system. The virtual machine is then automatically reverted to its original clean snapshot, preparing it for the next analysis cycle. This snapshot restoration ensures that no residual malware artifacts remain in the environment.

The collected logs are processed to generate structured analysis reports highlighting behavioral indicators of malicious activity. These reports assist in identifying whether the executed file demonstrates suspicious or harmful behavior. The complete process operates without requiring manual interaction, allowing multiple samples to be analyzed sequentially or in parallel.

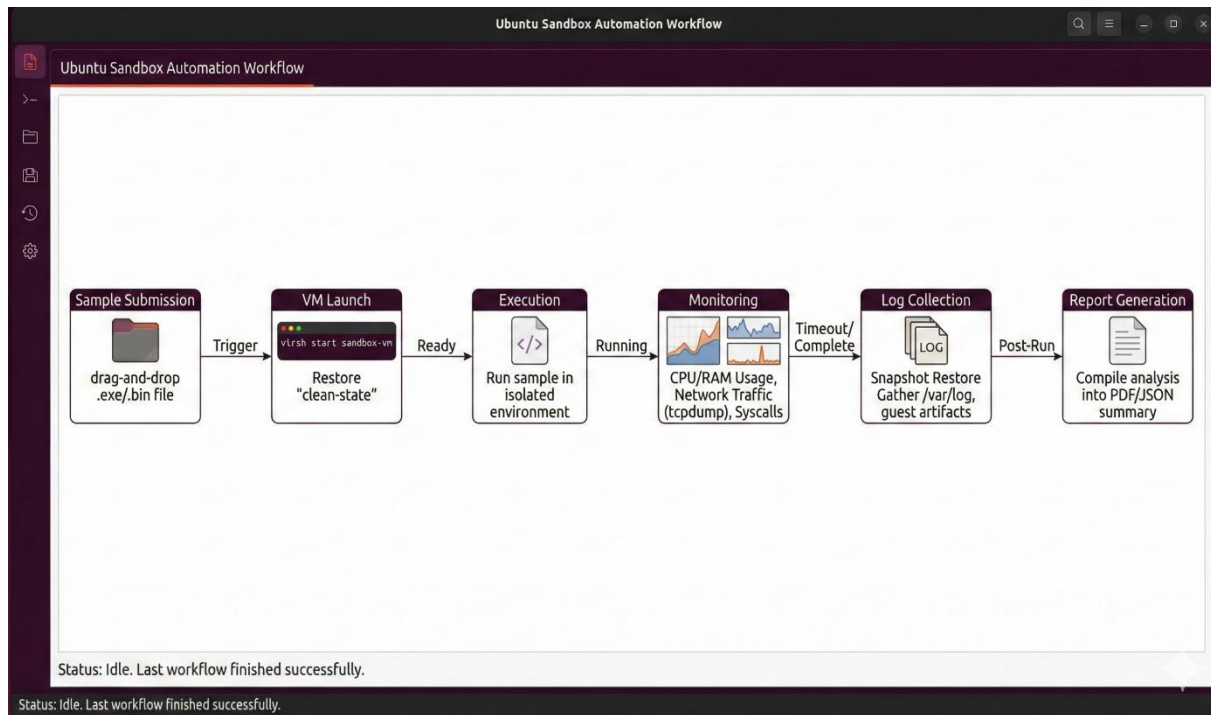


Figure 3.4.1 Ubuntu Sandbox Automation Workflow for Malware Analysis

This automated workflow improves analysis speed, reduces human error, and enables scalable malware investigation, forming a central component of the proposed sandbox-based detection system.

3.5 Sample Submission and Execution Process

The sample submission and execution process defines how suspicious executable files are introduced into the sandbox environment and analyzed in a controlled and automated manner. This process ensures that each sample is handled securely, executed consistently, and monitored effectively to obtain accurate behavioral data.

When a suspicious executable file is provided for analysis, it is first stored in a dedicated input directory on the host system. Basic validation checks, such as file type verification and naming conventions, are performed to ensure that only supported executable formats are submitted for analysis. Once validated, the automation controller queues the sample for execution.

The system then launches a guest virtual machine from a clean baseline snapshot. After the virtual machine completes its boot sequence, the submitted sample is automatically transferred from the host to the guest environment through a controlled and secure transfer mechanism.

Shared folders and direct clipboard access are disabled to prevent accidental leakage between host and guest systems.

Once the sample is successfully placed inside the guest operating system, the automation controller initiates its execution. The sample is run under normal user-level privileges to mimic realistic execution conditions. During this period, monitoring tools actively track all system-level and network-level activities triggered by the sample.

The execution is allowed to continue for a predefined observation period to ensure sufficient time for malware behavior to manifest. After the execution window ends, the automation controller signals the guest system to stop monitoring and prepares for log collection. The virtual machine is then reverted to its clean snapshot state, ensuring a fresh environment for subsequent analyses.

This structured submission and execution process guarantees consistent handling of malware samples, safe containment of threats, and reliable collection of behavioral evidence required for further detection and evaluation.

3.6 Behavioral Data Collection Mechanism

The behavioral data collection mechanism is responsible for capturing all relevant system and network activities generated by a suspicious executable during its execution in the sandbox environment. Accurate and comprehensive data collection is essential for understanding malware behavior and identifying indicators of malicious activity.

Monitoring agents installed inside the guest virtual machine continuously observe runtime events throughout the analysis period. These agents track critical system operations such as process creation and termination, file and directory modifications, registry access and alterations, memory usage patterns, and API calls invoked by the executing program. Additionally, network monitoring tools record outbound and inbound network connections, DNS queries, and data transmission attempts initiated by the sample.

The collected behavioral data is stored in structured log files, which include timestamps, process identifiers, system call details, and network activity records. This detailed logging ensures that every significant action performed by the executable is documented for further examination. To minimize interference with malware execution, the monitoring tools operate in the background with optimized resource usage.

Once the execution phase is completed, all recorded logs are securely transferred from the guest virtual machine to the host system. The data is then organized into standardized formats such as JSON or database entries to enable efficient processing and report generation. Redundant or irrelevant background system activities are filtered to focus on actions directly associated with the analyzed sample.

This behavioral data collection mechanism provides the foundation for identifying suspicious activity patterns. It ensures reliable evidence gathering for malware behavior analysis and supports the detection logic used to determine whether an executable demonstrates malicious characteristics.

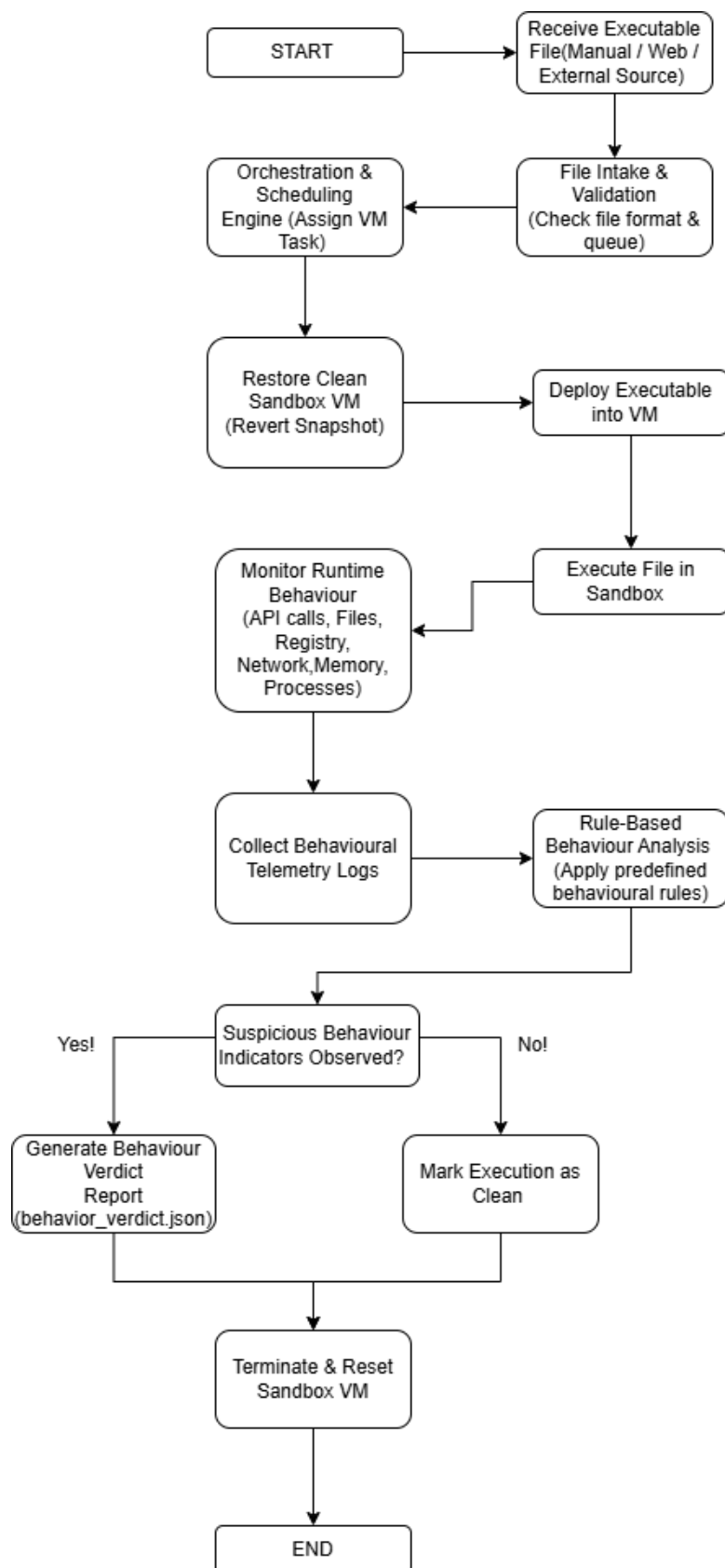


Figure 3.6 Workflow of the Proposed Automated Sandbox-Based Malware Analysis System

3.7 Log Processing and Feature Extraction

After behavioral data is collected from the sandbox environment, raw log files must be processed to extract meaningful information for malware detection. The log processing and feature extraction stage transforms large volumes of unstructured monitoring data into structured and relevant behavioral indicators.

The raw logs obtained from the monitoring agents contain extensive system and network activity records, including both legitimate background processes and actions performed by the analyzed sample. Therefore, an initial filtering step is applied to remove irrelevant system noise and isolate events directly associated with the suspicious executable. This improves accuracy and reduces unnecessary data volume.

Once filtered, the logs are parsed and categorized into different activity types such as process behavior, file system operations, registry modifications, and network communications. Each category is analyzed to extract specific features that may indicate malicious behavior. Examples of extracted features include creation of hidden files, modification of critical registry keys, unauthorized process injection attempts, repeated failed network connections, and access to sensitive system directories.

The extracted features are then structured into a standardized format suitable for report generation and detection analysis. This structured representation allows easy interpretation of behavioral patterns and supports automated evaluation of suspicious activity. Time-based correlation is also applied to understand the sequence of actions performed by the executable during runtime.

This log processing and feature extraction mechanism plays a crucial role in converting raw behavioral data into actionable insights. It forms the analytical bridge between sandbox monitoring and malicious behavior detection, enabling the system to identify potential threats efficiently and consistently.

3.8 Malicious Behaviour Detection Logic

The core objective of the proposed system is to detect malicious behaviour exhibited by executable files during runtime execution inside an isolated sandbox environment. Unlike conventional malware classification systems that rely on machine learning models for prediction, the proposed system adopts a rule-based behavioural detection approach. This

design choice ensures transparency, explainability, and deterministic interpretation of results, which is essential for behavioural malware analysis.

Once a suspicious executable file is submitted to the sandbox, it is executed inside a controlled virtual machine. During execution, the system continuously monitors multiple runtime activities including:

- System API calls and process creation events
- File system modifications
- Registry access and persistence attempts
- Memory manipulation patterns
- Network communication and domain connections

These behavioural events are collected and stored in structured log files generated by the sandbox environment. After execution completes, the automation controller triggers a behavioural parsing module, which reads the generated logs and extracts significant runtime indicators.

The detection logic then applies a predefined set of behavioural rules, where each rule corresponds to a known malicious activity pattern.

Examples of rule conditions include:

- Creation of hidden or system-level files
- Attempts to modify startup registry keys
- Injection into system processes
- Unusual memory allocation sequences
- Unauthorized outbound network communication

If one or more rule conditions are satisfied, the system flags the executable as exhibiting malicious behaviour. Otherwise, the executable is considered to have exhibited no significant malicious activity during observation.

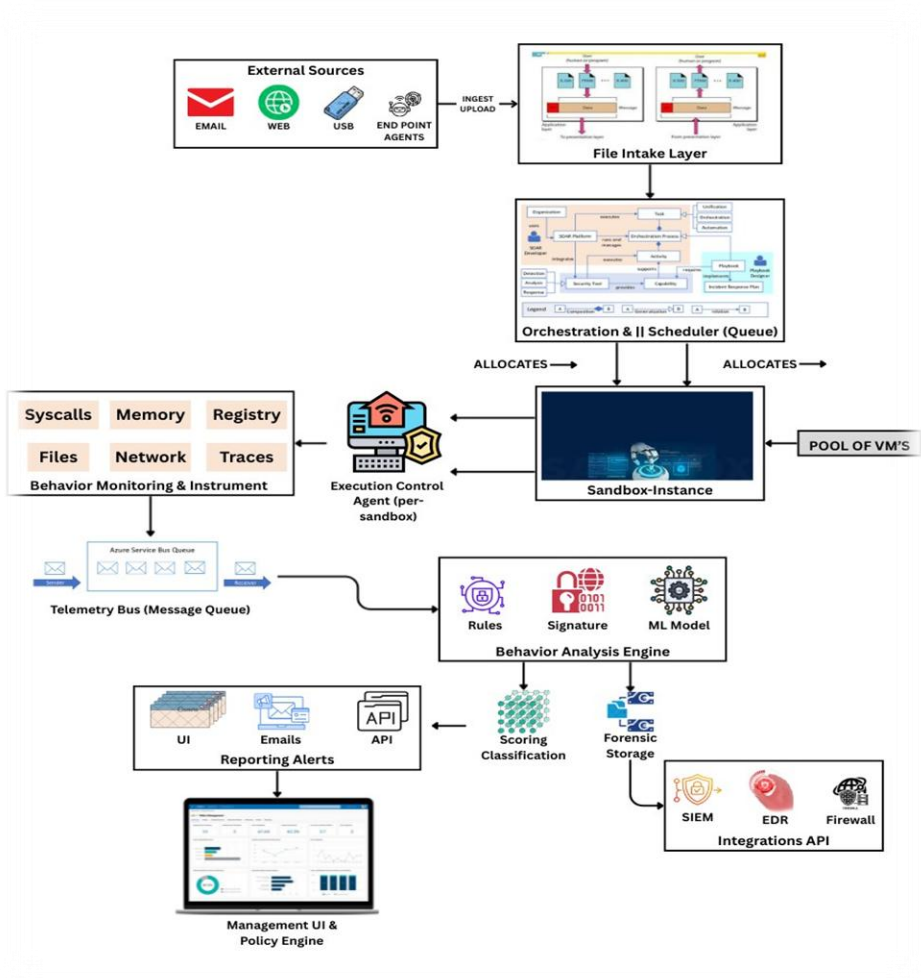


Figure 3.8 Architecture of the Proposed Automated Sandbox-Based Malware Analysis System

CHAPTER 4

IMPLEMENTATION & EXPERIMENTAL RESULTS

4.1 Tools and Technologies Used

The implementation of the automated sandbox-based malware analysis system requires a combination of software tools, frameworks, and technologies that enable virtualization, sandbox automation, behavioral monitoring, and report generation. The selected tools are chosen based on reliability, compatibility, open-source availability, and suitability for academic research purposes.

A virtualization platform is used to create and manage the isolated execution environment for malware samples. Virtualization software such as VirtualBox or VMware provides the capability to run guest operating systems, manage system snapshots, and configure network isolation. This ensures that malware samples can be executed safely without affecting the host system.

The core sandbox framework utilized in this project is Cuckoo Sandbox. It provides automated sample execution, virtual machine control, behavioral monitoring, and report generation features. Cuckoo allows customization of analysis workflows and integration with additional monitoring tools, making it suitable for building a fully automated malware analysis lab.

The host operating system is based on a Linux distribution, which offers stability, security, and strong support for scripting and automation. Python is used as the primary scripting language for automation control, log processing, and interaction with the sandbox framework. Python is chosen due to its extensive libraries and native compatibility with Cuckoo Sandbox.

The guest operating system installed in the virtual machine is a Windows platform, as most malware targets Windows environments. Monitoring agents and system tracking utilities are installed within the guest system to capture runtime behavioral data.

Additional tools such as network packet analyzers, process monitoring utilities, and log management tools are integrated to enhance behavioral data collection and analysis. Databases or structured file formats are used to store logs and reports for further evaluation.

Together, these tools and technologies create a robust foundation for implementing an automated sandbox-based malware behaviour detection system, ensuring safety, efficiency, and extensibility for future enhancements.

Table 4.1 Tools and Technologies Used in Implementation

Tool / Technology	Purpose
Cuckoo Sandbox	Automated malware analysis
VirtualBox	Virtual machine management
Python	Automation scripting
Wireshark	Network monitoring
Windows 10 VM	Malware execution

4.2 Implementation of Cuckoo Sandbox Environment

The implementation of the Cuckoo Sandbox environment forms the core operational component of the proposed malware analysis system. Cuckoo Sandbox is deployed on the host machine to manage virtual machines, automate malware execution, monitor runtime behavior, and generate comprehensive analysis reports. Proper implementation and configuration of Cuckoo ensure reliable and repeatable malware analysis.

The installation process begins by setting up a Linux-based host operating system with required dependencies, including Python, virtualization drivers, database services, and network configuration utilities. Cuckoo Sandbox is then installed and configured to interact with the virtualization platform, allowing automated control of guest virtual machines. The sandbox configuration includes defining virtual machine profiles, specifying snapshot names, execution time limits, and network settings.

A Windows-based guest virtual machine is prepared with Cuckoo's monitoring agent installed. This agent is responsible for tracking system-level activities inside the guest during malware execution. Once the virtual machine is fully configured, a clean baseline snapshot is created. This snapshot is used to restore the system before every new analysis, ensuring a consistent and contamination-free environment.

Cuckoo's analysis modules are configured to capture various behavioral aspects, including process activities, file system changes, registry modifications, memory operations, and network traffic. Reporting modules are enabled to generate structured output files in formats such as JSON and HTML, which summarize observed behaviors and extracted indicators.

Automation scripts are developed to manage sample submission, initiate analysis tasks, monitor execution status, retrieve reports, and reset the virtual environment. This automation eliminates manual intervention, enabling continuous and efficient processing of multiple malware samples.

IMPLEMENTATION & EXPERIMENTAL RESULTS

Through this implementation, Cuckoo Sandbox provides a flexible and powerful foundation for the proposed automated malware behaviour detection system, allowing controlled execution, detailed monitoring, and systematic reporting of suspicious executable files.

```
2025-12-27 16:14:55,005 [cuckoo.core.startup] DEBUG: -- WarbotURL
2025-12-27 16:14:55,005 [cuckoo.core.startup] DEBUG: -- Whitmo
2025-12-27 16:14:55,005 [cuckoo.core.startup] DEBUG: -- Win32ProcessCreate
2025-12-27 16:14:55,005 [cuckoo.core.startup] DEBUG: -- WineDetect
2025-12-27 16:14:55,005 [cuckoo.core.startup] DEBUG: -- WinSCP
2025-12-27 16:14:55,005 [cuckoo.core.startup] DEBUG: -- WinSxsBot
2025-12-27 16:14:55,005 [cuckoo.core.startup] DEBUG: -- WMIAntiVM
2025-12-27 16:14:55,005 [cuckoo.core.startup] DEBUG: -- WMIPersistence
2025-12-27 16:14:55,005 [cuckoo.core.startup] DEBUG: -- WMIService
2025-12-27 16:14:55,005 [cuckoo.core.startup] DEBUG: -- WormAllaple
2025-12-27 16:14:55,005 [cuckoo.core.startup] DEBUG: -- WormKolabc
2025-12-27 16:14:55,005 [cuckoo.core.startup] DEBUG: -- XenDetectKeys
2025-12-27 16:14:55,005 [cuckoo.core.startup] DEBUG: -- XtremeRAT
2025-12-27 16:14:55,005 [cuckoo.core.startup] DEBUG: -- Xworm
2025-12-27 16:14:55,005 [cuckoo.core.startup] DEBUG: -- Zegost
2025-12-27 16:14:55,005 [cuckoo.core.startup] DEBUG: -- ZeusMutexes
2025-12-27 16:14:55,005 [cuckoo.core.startup] DEBUG: -- ZeusP2P
2025-12-27 16:14:55,005 [cuckoo.core.startup] DEBUG: -- ZeusURL
2025-12-27 16:14:55,005 [cuckoo.core.startup] DEBUG: -- ZoneID
2025-12-27 16:14:55,005 [cuckoo.core.startup] DEBUG: Imported "reporting" modules:
2025-12-27 16:14:55,005 [cuckoo.core.startup] DEBUG: -- ElasticSearch
2025-12-27 16:14:55,005 [cuckoo.core.startup] DEBUG: -- Feedback
2025-12-27 16:14:55,005 [cuckoo.core.startup] DEBUG: -- JsonDump
2025-12-27 16:14:55,005 [cuckoo.core.startup] DEBUG: -- Mattermost
2025-12-27 16:14:55,005 [cuckoo.core.startup] DEBUG: -- MISP
2025-12-27 16:14:55,005 [cuckoo.core.startup] DEBUG: -- Moloch
2025-12-27 16:14:55,005 [cuckoo.core.startup] DEBUG: -- MongoDB
2025-12-27 16:14:55,006 [cuckoo.core.startup] DEBUG: -- Notification
2025-12-27 16:14:55,006 [cuckoo.core.startup] DEBUG: -- Singlefile
2025-12-27 16:14:55,009 [cuckoo.core.startup] DEBUG: Checking for locked tasks..
2025-12-27 16:14:55,015 [cuckoo.core.startup] DEBUG: Checking for pending service tasks..
2025-12-27 16:14:55,018 [cuckoo.core.startup] DEBUG: Initializing Yara...
2025-12-27 16:14:55,019 [cuckoo.core.startup] DEBUG: -- binaries embedded.yar
2025-12-27 16:14:55,019 [cuckoo.core.startup] DEBUG: -- binaries filetype.yar
2025-12-27 16:14:55,019 [cuckoo.core.startup] DEBUG: -- binaries shellcodes.yar
2025-12-27 16:14:55,021 [cuckoo.core.startup] DEBUG: -- binaries vmdetect.yar
2025-12-27 16:14:55,021 [cuckoo.core.startup] DEBUG: -- scripts applocker_bypass.yar
2025-12-27 16:14:55,021 [cuckoo.core.startup] DEBUG: -- scripts powerfun.yar
2025-12-27 16:14:55,021 [cuckoo.core.startup] DEBUG: -- scripts powershell_AMSI.yar
2025-12-27 16:14:55,021 [cuckoo.core.startup] DEBUG: -- scripts powershell_BITS_transfer.yar
2025-12-27 16:14:55,021 [cuckoo.core.startup] DEBUG: -- scripts powershell_ddl_rc4.yar
2025-12-27 16:14:55,021 [cuckoo.core.startup] DEBUG: -- scripts powershell_dfsp.yar
2025-12-27 16:14:55,021 [cuckoo.core.startup] DEBUG: -- scripts powershell_di.yar
2025-12-27 16:14:55,021 [cuckoo.core.startup] DEBUG: -- scripts powershell_empire.yar
2025-12-27 16:14:55,021 [cuckoo.core.startup] DEBUG: -- scripts powershell_meterpreter.yar
2025-12-27 16:14:55,021 [cuckoo.core.startup] DEBUG: -- scripts powershell_txt_c2.yar
2025-12-27 16:14:55,021 [cuckoo.core.startup] DEBUG: -- scripts powershell_unicorn.yar
2025-12-27 16:14:55,022 [cuckoo.core.startup] DEBUG: -- scripts powerworm.yar
2025-12-27 16:14:55,022 [cuckoo.core.startup] DEBUG: -- shellcode metasploit.yar
2025-12-27 16:14:55,022 [cuckoo.core.startup] DEBUG: -- office dde.yar
2025-12-27 16:14:55,022 [cuckoo.core.startup] DEBUG: -- office ole.yar
2025-12-27 16:14:55,024 [cuckoo.core.scheduler] INFO: Using "virtualbox" as machine manager
2025-12-27 16:14:55,424 [cuckoo.core.scheduler] INFO: Loaded 1 machine/s
2025-12-27 16:14:55,428 [cuckoo.core.scheduler] INFO: Waiting for analysts tasks.
```

Figure 4.2 Cuckoo Sandbox Core Startup and Module Initialization

Figure 4.2 shows the startup process of the Cuckoo Sandbox core engine on the host system. The terminal output confirms that analysis modules, reporting modules, and YARA rule sets are successfully loaded. It also indicates that the VirtualBox machine manager is detected and that configured virtual machines are available for sandbox execution. The final scheduler message shows that the sandbox is actively running and waiting for analysis tasks. This verifies that the Cuckoo Sandbox environment is correctly installed, configured, and operational for automated malware analysis.

4.3 Automation Script Development

Automation script development is a crucial part of the proposed system, as it enables fully automated control of the sandbox environment and eliminates the need for manual intervention during malware analysis. These scripts coordinate sample submission, virtual machine

management, execution monitoring, log collection, and environment restoration in a structured workflow.

Python is used as the primary scripting language for automation due to its strong compatibility with Cuckoo Sandbox and availability of extensive libraries for system control and data processing. Custom scripts are developed to interface with Cuckoo's API and command-line utilities to initiate analysis tasks automatically when a new executable sample is placed in the input directory.

The automation scripts perform several key functions. First, they detect newly submitted samples and register them with the sandbox for analysis. Next, they instruct Cuckoo to launch the appropriate virtual machine snapshot and execute the submitted file inside the guest environment. The scripts monitor the execution status and ensure that each sample runs for the predefined observation period.

After execution is completed, the automation scripts automatically retrieve generated behavioral reports and store them in a structured location for further processing. They then trigger restoration of the virtual machine to its clean baseline snapshot, preparing the environment for the next analysis cycle. Error-handling routines are included to manage failed executions, system crashes, or incomplete analyses.

Additionally, automation scripts assist in post-processing tasks such as filtering raw logs, extracting behavioral features, and generating summarized detection reports. This automated pipeline ensures consistency in analysis, reduces processing time, and allows multiple malware samples to be handled efficiently.

Through automation script development, the system achieves continuous, repeatable, and scalable malware analysis, forming a key component of the implementation phase of the project.

4.4 Dataset and Sample Collection

The effectiveness of any malware analysis system depends on the quality and diversity of samples used for evaluation. In this project, a dataset of executable files is collected to test and validate the performance of the proposed automated sandbox-based malware behaviour detection system. The dataset includes both malicious and benign executable files to ensure balanced and realistic assessment.

Malware samples are obtained from publicly available cybersecurity research repositories and malware sharing platforms that provide controlled access for academic and research purposes. These samples include various malware families such as trojans, ransomware, spyware, and worms, representing different attack behaviors and execution patterns. Each collected sample is handled with strict security precautions and stored in encrypted archives to prevent accidental execution outside the sandbox environment.

Benign executable files are also included in the dataset to evaluate the system's ability to distinguish legitimate software from malicious programs and to assess the rate of false positive detections. These benign samples consist of commonly used trusted applications and system utilities.

Before analysis, all samples are labeled and organized based on type and source. Basic static verification such as file hashing is performed to maintain sample integrity and prevent duplicate entries. The dataset is then fed into the sandbox environment through the automated sample submission process.

This dataset and sample collection approach provides a realistic testing environment for the proposed system. It enables evaluation of behavioral detection effectiveness across diverse malware categories and ensures meaningful performance analysis of the automated sandbox framework.

4.5 Experiment Execution Procedure

The experiment execution procedure defines the systematic process followed to evaluate the performance of the proposed automated sandbox-based malware analysis system. A structured execution procedure ensures consistent testing conditions, reliable data collection, and accurate performance assessment.

The experiment begins with preparing the sandbox environment by restoring the guest virtual machine to its clean baseline snapshot. This guarantees that each executable sample is analyzed in an identical and uncontaminated system state. Once the environment is ready, a sample from the prepared dataset is submitted to the automation controller for analysis.

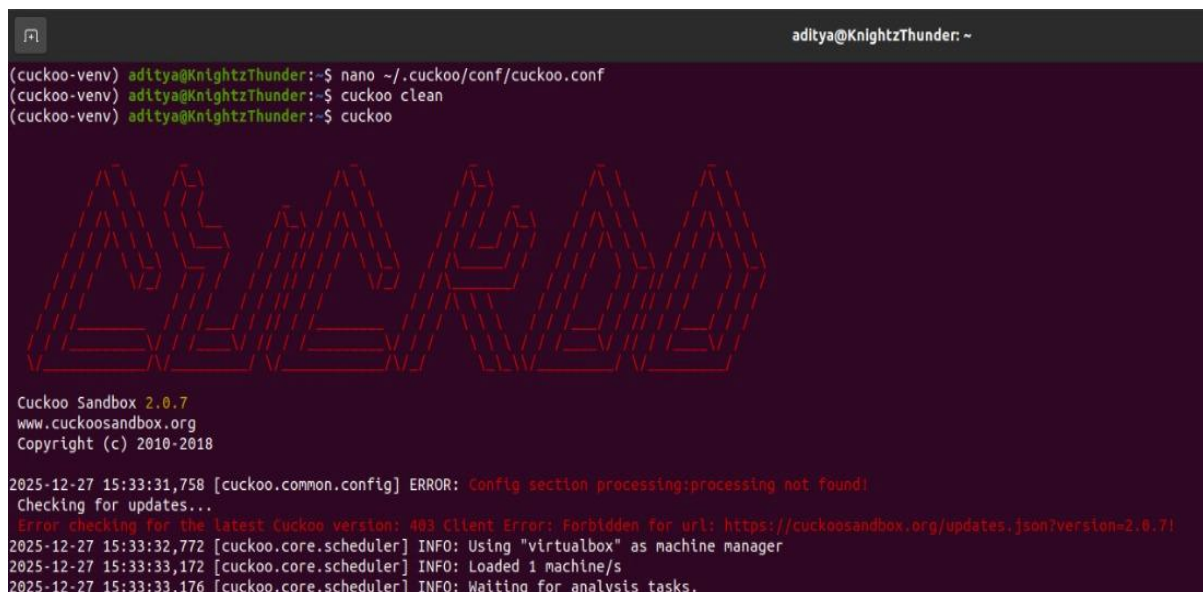
Upon submission, the automation scripts initiate the execution of the sample inside the sandboxed virtual machine. The sample is allowed to run for a predefined observation period, during which all system-level and network-level activities are continuously monitored.

Behavioral data such as process creation, file operations, registry modifications, memory activity, and network communications are recorded by monitoring agents.

After completion of the execution window, the system automatically stops monitoring and collects all generated behavioral logs and reports from the guest machine. These reports are stored on the host system for further log processing and feature extraction. The virtual machine is then reverted to the clean snapshot state, preparing it for the next sample execution.

This process is repeated for each sample in the dataset to ensure uniform testing conditions across all experiments. Any failed executions or incomplete analyses are logged separately and reprocessed if required. The collected reports from all experiments form the basis for behavioral analysis, detection logic evaluation, and performance measurement.

The experiment execution procedure ensures controlled testing, consistent data acquisition, and reproducible results, providing a reliable foundation for evaluating the effectiveness of the proposed malware behaviour detection system.

A terminal window titled 'aditya@KnightzThunder: ~' showing the execution of Cuckoo Sandbox. The user runs 'nano ~/.cuckoo/conf/cuckoo.conf', 'cuckoo clean', and 'cuckoo'. The output displays the Cuckoo Sandbox logo, version 2.0.7, website, and copyright. It then shows log messages: an error for config processing, a check for updates (failing due to a forbidden URL), and successful messages for using VirtualBox as a machine manager, loading 1 machine, and waiting for analysis tasks.

```
aditya@KnightzThunder: ~  
(cuckoo-venv) aditya@KnightzThunder:~$ nano ~/.cuckoo/conf/cuckoo.conf  
(cuckoo-venv) aditya@KnightzThunder:~$ cuckoo clean  
(cuckoo-venv) aditya@KnightzThunder:~$ cuckoo  
  
Cuckoo Sandbox 2.0.7  
www.cuckoosandbox.org  
Copyright (c) 2010-2018  
  
2025-12-27 15:33:31,758 [cuckoo.common.config] ERROR: Config section processing:processing not found!  
Checking for updates...  
Error checking for the latest Cuckoo version: 403 Client Error: Forbidden for url: https://cuckoosandbox.org/updates.json?version=2.0.7!  
2025-12-27 15:33:32,772 [cuckoo.core.scheduler] INFO: Using "virtualbox" as machine manager  
2025-12-27 15:33:33,172 [cuckoo.core.scheduler] INFO: Loaded 1 machine/s  
2025-12-27 15:33:33,176 [cuckoo.core.scheduler] INFO: Waiting for analysis tasks.
```

Figure 4.5.1 Cuckoo Sandbox Startup and Configuration Initialization

Figure 4.5.1 shows the initialization and startup of the Cuckoo Sandbox framework on the host system. The terminal output indicates that configuration files are loaded, the Cuckoo core engine is started, and the virtualization platform (VirtualBox) is successfully detected as the machine manager. The log further confirms that one virtual machine is loaded and the scheduler is actively waiting for analysis tasks.

A configuration warning related to a missing processing section and an update check error are displayed; however, these warnings do not affect the core functioning of the sandbox. The subsequent messages verify that the sandbox environment is correctly initialized and ready to accept malware analysis tasks. This confirms that the Cuckoo Sandbox installation and configuration are successfully completed for implementing the automated malware analysis environment.

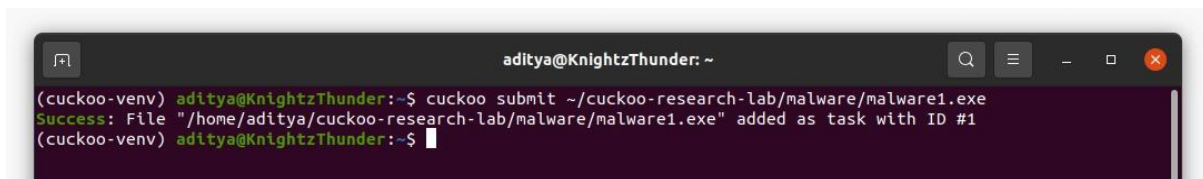


Figure 4.5.2 Malware Sample Submission to Cuckoo Sandbox

Figure 4.5.2 illustrates the submission of a malware sample to the Cuckoo Sandbox framework. The terminal output shows the execution of the `cuckoo submit` command, where the suspicious executable file `malware1.exe` is successfully added to the sandbox as a new analysis task with a unique task ID. This confirms that the sample submission mechanism is functioning correctly and that the sandbox automation pipeline is ready to initiate the analysis process. The successful task registration demonstrates that the system can accept executable samples for further automated execution and behavioral monitoring.

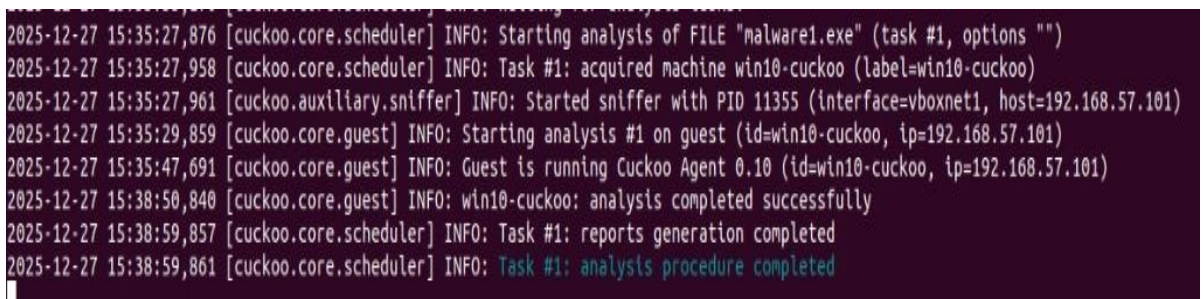
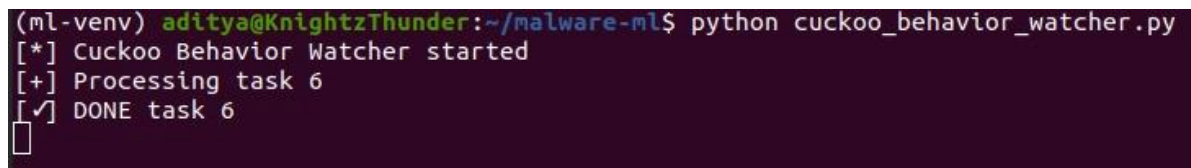


Figure 4.5.3 Malware Sample Execution and Report Generation Log

Figure 4.5.3 shows the real-time execution and completion of a malware analysis task inside the Cuckoo Sandbox environment. The log indicates that the Cuckoo scheduler successfully starts the analysis of the submitted malware sample and acquires the configured Windows virtual machine. It confirms the activation of the network sniffer for monitoring

communication behavior and the startup of the Cuckoo Agent inside the guest operating system.

The final log messages verify that the malware sample execution completed successfully and that the sandbox generated behavioral analysis reports. This confirms that the sandbox environment correctly manages virtual machine control, sample execution, runtime monitoring, and report generation as part of the automated malware analysis implementation.

A terminal window with a dark purple background. The prompt is '(ml-venv) aditya@KnightzThunder:~/malware-ml\$'. The command 'python cuckoo_behavior_watcher.py' has been executed. The output shows four lines: '[*] Cuckoo Behavior Watcher started', '[+] Processing task 6', '[✓] DONE task 6', and a cursor on the next line.

```
(ml-venv) aditya@KnightzThunder:~/malware-ml$ python cuckoo_behavior_watcher.py
[*] Cuckoo Behavior Watcher started
[+] Processing task 6
[✓] DONE task 6
█
```

Figure 4.5.4 Automated Behavior Report Extraction Script Execution

Figure 4.5.4 shows the execution of the automated behavioral watcher script developed for the proposed system. The script continuously monitors the Cuckoo Sandbox analysis storage directory and detects when a malware analysis task is completed. Once a task finishes, the script automatically processes the generated sandbox reports and extracts relevant behavioral information.

The log output confirms that the behavior watcher successfully identifies a completed analysis task and completes the post-processing step without manual intervention. This automation ensures that behavioral data is consistently collected and structured for further rule-based malicious behaviour detection. It represents an important component of the overall automation pipeline of the implemented sandbox-based malware analysis system.

```

aditya@KnightzThunder:~/cuckoo/storage/analyses$ cd 6
aditya@KnightzThunder:~/cuckoo/storage/analyses/6$ ls
analysis.log      cuckoo.log      files           reboot.json    tlsmaster.txt
behavior_verdict.json  dump.pcap      logs           reports
binary           dump_sorted.pcap  memory         shots
buffer           extracted        network        task.json
aditya@KnightzThunder:~/cuckoo/storage/analyses/6$ cat behavior_verdict.json
{
  "verdict": "malicious_behavior_detected",
  "severity": "medium",
  "behaviors_detected": [
    "evasion",
    "memory_staging",
    "network"
  ],
  "scores": {
    "evasion": 2,
    "crypto": 0,
    "memory_staging": 1,
    "persistence": 0,
    "network": 1
  },
  "evidence": {
    "signatures": [
      "recon_fingerprint",
      "pe_features",
      "injection_process_search",
      "antivm_network_adapters"
    ],
    "api_calls": [
      "5544"
    ],
    "processes": [
      "lsass.exe",
      "malware1.exe"
    ],
    "domains": [
      "DESKTOP-2UN30B8.local"
    ]
  }
}
aditya@KnightzThunder:~/cuckoo/storage/analyses/6$ █

```

Figure 4.5.5 Generated Behavioral Verdict File After Rule-Based Detection

Figure 4.5.5 shows the final behavioral verdict file generated after processing the sandbox analysis reports. The screenshot displays the contents of the `behavior_verdict.json` file stored inside the Cuckoo Sandbox analysis directory. This file contains the final detection result produced by the rule-based behavioral analysis engine developed in this project.

The output indicates that malicious behavior has been detected, along with the severity level. It also lists the specific behavioral indicators observed during runtime, such as evasion techniques, memory staging, and network activity. Additionally, the file records structured evidence including triggered detection signatures, monitored API calls, involved processes, and accessed system domains.

This result confirms that the implemented system successfully extracts behavioral data from sandbox reports and applies rule-based logic to identify malicious activities. It represents the final automated detection output of the proposed sandbox-based malware behaviour analysis system.

4.6 Observed Behavioral Patterns

During the execution of malware samples in the sandbox environment, a variety of behavioral patterns are observed and recorded. These patterns provide insight into the actions performed by executable files at runtime and serve as key indicators for identifying malicious activity. By analyzing these behaviors, the system can distinguish between benign and malicious software based on their operational characteristics.

One commonly observed pattern is suspicious process activity, where malware creates or injects code into new processes to hide execution or gain elevated privileges. Some samples spawn multiple background processes to maintain persistence or avoid termination. Abnormal process hierarchies and repeated process creation attempts are strong indicators of malicious intent.

File system modifications are also frequently observed. Malware often creates hidden files, modifies system directories, or drops additional payload files during execution. Attempts to overwrite critical system files or create executable files in temporary directories indicate potential infection behavior.

Registry modifications are another significant behavioral pattern. Many malware samples alter registry keys to achieve persistence, enabling automatic execution upon system startup. Unauthorized changes to security-related registry entries are treated as suspicious activities.

Network communication behavior provides crucial evidence of external interaction. Observed patterns include repeated attempts to contact remote servers, suspicious DNS queries, data transmission to unknown IP addresses, and connection to command-and-control infrastructures. These behaviors indicate possible data exfiltration or remote attacker control.

Security evasion actions are also detected in some samples. These include attempts to disable antivirus services, stop system monitoring tools, or detect virtualized environments. Such actions demonstrate efforts to avoid detection and analysis.

By compiling these observed behavioral patterns, the system creates a behavioral profile for each analyzed sample. These profiles are used by the detection logic to assess the likelihood of malicious activity and support final classification results in the analysis reports.

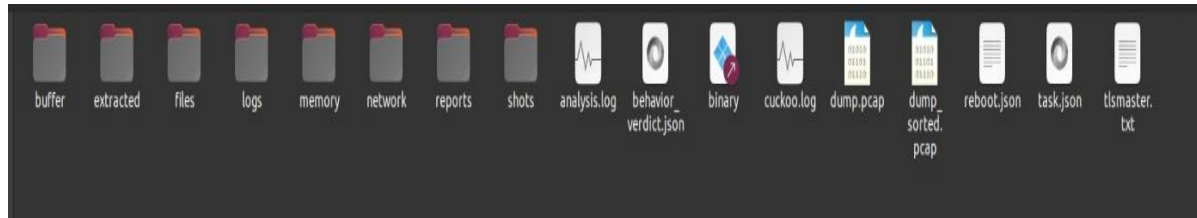


Figure 4.6 Cuckoo Sandbox Generated Analysis Artifacts Directory

Figure 4.6 shows the analysis results directory generated by the Cuckoo Sandbox after executing a malware sample. The directory contains multiple artifacts such as system logs, extracted files, network packet captures, memory dumps, screenshots, and detailed analysis reports. The presence of the behavioral verdict file further confirms that post-processing of sandbox reports has been completed successfully.

These generated artifacts provide comprehensive runtime evidence of system and network activities performed by the analyzed executable. They form the basis for behavioral inspection and rule-based malicious behaviour detection in the proposed system.

4.7 Result Analysis and Detection Accuracy

After executing the collected samples in the sandbox environment, the generated behavioral reports are analyzed to understand the runtime activities of each executable. The purpose of result analysis in this project is to examine observed behavior patterns and determine whether the system successfully captures meaningful indicators of malicious activity.

Each executed sample produces a detailed report containing system-level events such as process creation, file system changes, registry modifications, and network communications. These reports are manually and programmatically reviewed to identify suspicious actions commonly associated with malware, such as unauthorized persistence mechanisms, hidden file creation, abnormal process injection, and external server communication attempts.

For malicious samples, the sandbox environment successfully records clear behavioral evidence of harmful activities. For benign samples, reports primarily show normal execution behavior without suspicious system modifications or network anomalies. This distinction

demonstrates that the sandbox setup and monitoring components are functioning correctly and capturing relevant behavioral information.

Rather than providing statistical classification accuracy, the focus of this project is on validating that the automated system can reliably execute samples, collect comprehensive behavioral logs, and present meaningful analysis reports that support malware identification. The results confirm that the proposed system achieves its objective of automated behavior-based malware analysis.

4.8 Performance Evaluation

Performance evaluation in this project focuses on the operational efficiency and reliability of the automated sandbox environment rather than predictive detection accuracy. The evaluation examines how effectively the system executes malware samples, collects behavioral data, restores clean environments, and handles repeated analysis cycles.

During experimentation, the automation workflow successfully manages virtual machine startup, sample execution, monitoring, report collection, and snapshot restoration without manual intervention. The system consistently maintains isolation between host and guest environments, ensuring safe malware execution and preventing system contamination.

Execution time per sample remains stable across multiple runs, indicating reliable automation and efficient resource management. The host system effectively handles virtualization processes, log storage, and report generation without performance degradation or system crashes.

Overall, the performance evaluation confirms that the proposed system provides a stable, repeatable, and fully automated sandbox environment for malware behavior analysis. The framework meets the project goal of automating malware execution and behavior monitoring and provides a solid foundation for future extensions such as advanced detection logic or machine learning-based classification.

CHAPTER 5

CONCLUSION AND FUTURE ENHANCEMENTS

5.1 Conclusion

This project presented the design and implementation of an automated sandbox-based system for detecting malicious behaviour in executable files through dynamic analysis and behavioral monitoring. The primary focus was to build a secure and isolated environment where suspicious files could be executed safely, their runtime activities observed, and behavioral evidence collected for malware identification.

The work began with studying existing malware detection techniques, including static, dynamic, and sandbox-based analysis methods. Based on this understanding, a malware analysis lab environment was designed using virtualization technology to ensure controlled execution and system isolation. Cuckoo Sandbox was implemented as the core framework to manage virtual machine execution, monitoring, and report generation.

Automation scripts were developed to handle sample submission, virtual machine control, execution scheduling, behavioral data collection, and environment restoration. This automation enabled continuous and repeatable analysis of executable samples without manual intervention. Behavioral logs generated during execution were processed to extract relevant activity patterns such as file system modifications, registry changes, process behavior, and network communications.

A dataset consisting of malicious and benign executable files was used to validate the functionality of the system. Experimental execution confirmed that the sandbox environment successfully executed samples, recorded meaningful behavioral data, and produced detailed analysis reports. The system demonstrated reliable automation, safe containment of malware, and consistent behavioral monitoring across multiple analysis cycles.

Overall, the project successfully achieved its objective of building a functional and automated sandbox-based malware behaviour analysis framework. The developed system provides a practical foundation for behavioral malware investigation in academic and research environments and establishes a base for future enhancements in advanced threat detection.

5.4 Proposed Future Enhancements

While the proposed automated sandbox-based malware behaviour analysis system successfully demonstrates safe execution and behavioral monitoring of suspicious executable files, several enhancements can be considered to improve its effectiveness, scalability, and analytical depth in future developments.

One potential enhancement is the integration of advanced behavioral classification techniques to assist in automated decision-making. Currently, the system provides rule-based behavioral identification; in the future, intelligent classification mechanisms can be added to automatically categorize malware families based on observed activity patterns.

The sandbox environment can be further improved to address sandbox evasion techniques used by sophisticated malware. This includes making the virtual machine environment more realistic by simulating user activity, installing common software applications, and masking virtualization artifacts to reduce detection by malware.

Another important enhancement is extending support for additional malware formats, such as script-based files, document-based macro malware, and mobile application packages. This would broaden the system's analytical capability beyond Windows executable files.

The current framework can also be scaled to support parallel analysis of multiple samples by deploying multiple sandbox instances running simultaneously. This would significantly improve processing speed when handling large malware datasets.

In addition, integrating threat intelligence feeds and external reputation services can enrich analysis reports with contextual information about known malicious domains, IP addresses, and malware signatures.

Finally, improving the reporting interface with visual behavioral graphs and dashboards would make analysis results easier to interpret for security researchers and incident response teams.

These future enhancements will strengthen the automation, realism, and analytical power of the proposed sandbox-based malware analysis system, making it more adaptable to evolving cyber threat landscapes.

References

- [1] Herrera-Silva, J. A., & Hernández-Álvarez, M. (2023). Dynamic feature dataset for ransomware detection using machine learning algorithms. *Sensors*, 23(3), 1053.
- [2] Almomani, O., Arabiat, A., Al Tayeb, M., Almaiah, M. A., Obeidat, M., Aldhyani, T. H., & Rowad, M. (2025). A robust model for Android malware detection via ML and DL classifiers. *Mesopotamian Journal of Big Data*, 2025, 261–277.
- [3] Pathak, A., Barman, U., & Kumar, T. S. (2025). Machine learning approach to detect Android malware using feature selection based on feature importance score. *Journal of Engineering Research*, 13(2), 712–720.
- [4] Denham, B., & Thompson, D. R. (2022, October). Ransomware and malware sandboxing. In *2022 IEEE 13th Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON)* (pp. 0173–0179). IEEE.
- [5] Yang, Z., & Xiao, J. (2024). A dynamic sandbox detection technique in a private cloud environment. *Scalable Computing: Practice and Experience*, 25(6), 4995–5004.
- [6] Lee, J., Kim, J., Jeong, H., & Lee, K. (2025). A machine learning-based ransomware detection method for attackers' neutralization techniques using format-preserving encryption. *Sensors*, 25(8), 2406.
- [7] Ilić, S., Gnjatović, M., Tot, I., Jovanović, B., Maček, N., & Gavrilović Božović, M. (2024). Going beyond API calls in dynamic malware analysis: A novel dataset. *Electronics*, 13(17), 3553. <https://doi.org/10.3390/electronics13173553>
- [8] Kamal, A., Derbali, M., Jan, S., Bangash, J. I., Khan, F. Q., Jerbi, H., & Ahmad, G. (2021). A user-friendly model for ransomware analysis using sandboxing. *Computers, Materials & Continua*, 67(3).
- [9] Mogicato, R., & Zermin, A. (2023). Design and implementation of a collaborative lightweight malware analysis sandbox using container virtualization. Universität Zürich, Zürich, Switzerland, Technical Report.
- [10] Kuchler, A., Mantovani, A., Han, Y., Bilge, L., & Balzarotti, D. (2021, February). Does every second count? Time-based evolution of malware behavior in sandboxes. In *NDSS 2021, Network and Distributed Systems Security Symposium*. Internet Society.

- [11] UPPIN, C. (2019). Dynamic Analysis of a Window-Based Malware Using Automated sandboxing. UPPIN, C. Dynamic Analysis of a Window-Based Malware Using Automated sandboxing.
- [12] Afianian, A., Niksefat, S., Sadeghiyan, B., & Baptiste, D. (2019). Malware dynamic analysis evasion techniques: A survey. *ACM Computing Surveys (CSUR)*, 52(6), 1-28.
- [13] Holm, K. S., Kvalvåg, M., Fauskrud, N., & Hoggen, O. H. (2019). Automated Malware Analysis Platform (Bachelor's thesis, NTNU).
- [14] Mills, A., & Legg, P. (2020). Investigating anti-evasion malware triggers using automated sandbox reconfiguration techniques. *Journal of Cybersecurity and Privacy*, 1(1), 19-39.
- [15] Choi, S., Bae, J., Lee, C., Kim, Y., & Kim, J. (2020). Attention-based automated feature extraction for malware analysis. *sensors*, 20(10), 2893.
- [16] Jamalpur, S., Navya, Y. S., Raja, P., Tagore, G., & Rao, G. R. K. (2018, April). Dynamic malware analysis using cuckoo sandbox. In *2018 Second international conference on inventive communication and computational technologies (ICICCT)* (pp. 1056-1060). IEEE.
- [17] Oktavianto, D., & Muhandianto, I. (2013). *Cuckoo malware analysis*. Birmingham: Packt Publishing.
- [18] Vasilescu, M., Gheorghe, L., & Tapus, N. (2014, September). Practical malware analysis based on sandboxing. In *2014 RoEduNet Conference 13th Edition: Networking in Education and Research Joint Event RENAM 8th Conference* (pp. 1-6). IEEE.
- [19] Wang, L., Wang, B., Liu, J., Miao, Q., & Zhang, J. (2019). Cuckoo-based malware dynamic analysis. *International Journal of Performability Engineering*, 15(3), 772.
- [20] Kaur, N., Bindal, A. K., & PhD, A. (2016). A complete dynamic malware analysis. *International Journal of Computer Applications*, 135(4), 20-25.
- [21] Pu, C., & Wan, L. (2026). APT Malware Detection Model Based on Heterogeneous Multimodal Semantic Fusion. *Applied Sciences*, 16(2), 1083.
- [22] Ahmed, H., Ali, H., Batool, K., Islam, M., & Irshad, A. (2026). A Comparative Analysis of Dynamic and Static Malware Detection Tools. In *AI-Driven Security for Next-Generation IoT Systems* (pp. 85-105). Cham: Springer Nature Switzerland.

- [23] Malipeddi, S. (2026). Dynamic Malware Analysis and Model Interpretability: Leveraging Hierarchical. *Advancement in Cyber Security and Digital Forensics: Proceedings of ICACSDF 2025*, 55.
- [24] Ahmad, A., Saif, A., & Alnagi, E. (2025, September). Detecting Malware in Portable Executable Files based on Reverse Engineering. In *2025 3rd International Conference on Artificial Intelligence, Blockchain, and Internet of Things (AIBThings)* (pp. 1-4). IEEE.
- [25] Malipeddi, S. (2026). Dynamic Malware Analysis and Model Interpretability: Leveraging Hierarchical. *Advancement in Cyber Security and Digital Forensics: Proceedings of ICACSDF 2025*, 55.